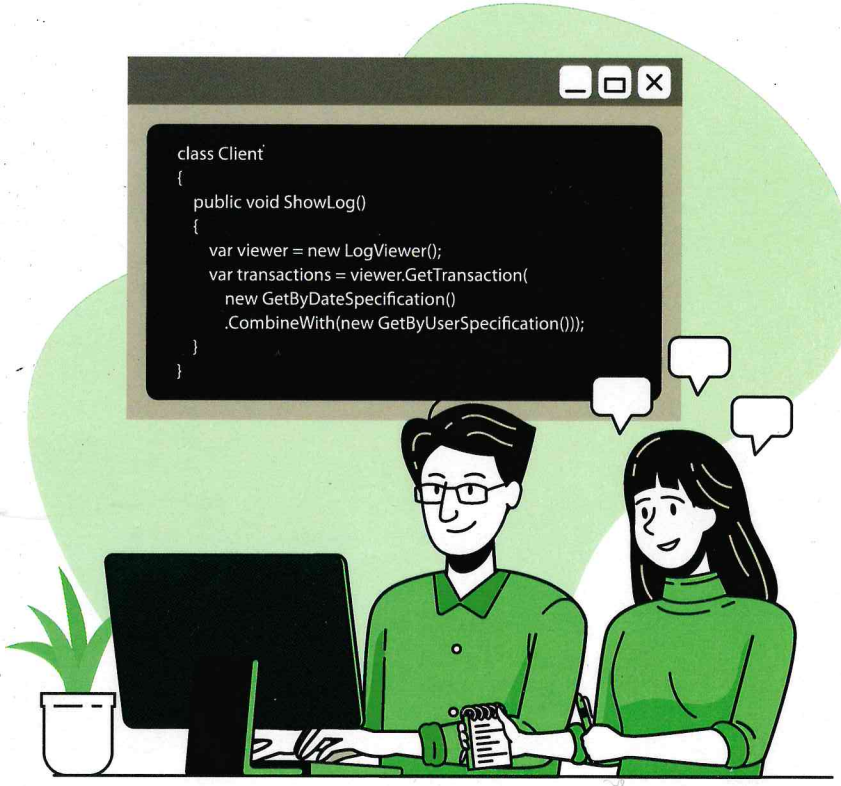
РЭУ.РФ

РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В. ПЛЕХАНОВА

Ю. П. Титов

ТЕХНОЛОГИИ КОЛЛЕКТИВНОЙ РАЗРАБОТКИ ИНФОРМАЦИОННЫХ СИСТЕМ



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Российский экономический университет имени Г. В. Плеханова»
(ФГБОУ ВО «РЭУ им. Г. В. Плеханова»)

Ю. П. Титов

ТЕХНОЛОГИИ КОЛЛЕКТИВНОЙ РАЗРАБОТКИ ИНФОРМАЦИОННЫХ СИСТЕМ

Учебное пособие

Москва
ФГБОУ ВО «РЭУ им. Г. В. Плеханова»
2024

УДК 004.41(075.8)
ББК 32.973.4я73
Т454

Рецензенты:
д-р экон. наук, проф. А. И. Уринцов (РЭУ им. Г. В. Плеханова);
д-р техн. наук В. А. Судаков (ИПМ им. М. В. Келдыша РАН)

Титов, Ю. П.

Т454 Технологии коллективной разработки информационных систем : учебное пособие / Ю. П. Титов. – Москва : ФГБОУ ВО «РЭУ им. Г. В. Плеханова», 2024. – 96 с.

ISBN 978-5-7307-2212-5

В учебном пособии отражены основные подходы, методы и технологии коллективной разработки информационных систем. Пособие посвящено вопросам разработки программного обеспечения командой разработчиков. В книге рассматриваются: управление процессами командной разработки, работа с системами контроля версий, проектирование архитектуры программного обеспечения, а также рефакторинг кода, code review и CI/CD-процессы. Помимо обширного теоретического материала, учебное пособие снабжено заданиями для практической и творческой работы.

Для студентов, обучающихся по направлению «Прикладная информатика».

УДК 004.41(075.8)
ББК 32.973.4я73

ISBN 978-5-7307-2212-5

© ФГБОУ ВО «РЭУ им. Г. В. Плеханова», 2024

СОДЕРЖАНИЕ

Предисловие 5

Глава 1. МЕТОДОЛОГИИ УПРАВЛЕНИЯ ПРОЦЕССОМ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 7
1.1. Водопадная модель (waterfall model) 7
1.2. Итерационная и спиральная модели 8
1.3. Модель прототипирования 10
1.4. V-образная модель разработки 10
1.5. Гибкие методологии разработки программного обеспечения 11

Глава 2. СИСТЕМЫ УПРАВЛЕНИЯ ПРОЕКТАМИ 16

Глава 3. СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ 19
3.1. Понятие и виды систем контроля версий 19
3.2. Системы контроля версий. Примеры 21
3.3. Система контроля версий Git 26

Глава 4. АРХИТЕКТУРЫ 32
4.1. Основные принципы создания архитектур. Универсальные критерии правильной архитектуры 32
4.2. Декомпозиция – основа архитектуры 35
4.3. Функциональная и иерархическая декомпозиции 35
4.4. Ослабление связанности между модулями. Decoupling 37
4.5. Замена прямых зависимостей 38
4.6. Закон Деметры (law of Demeter) 44
4.7. SOLID 45
4.7.1. Single Responsibility Principle (принцип единственной ответственности) 45
4.7.2. Open/Closed Principle (принцип открытия/закрытия) 47
4.7.3. Liskov Substitution Principle (принцип замещения Лисков) 48
4.7.4. Interface Segregation Principle (принцип изоляции интерфейса) 50
4.7.5. Dependency Inversion Principle (принцип обращения зависимости) 52

Глава 5. CI/CD 55

Глава 6. РЕФАКТОРИНГ КОДА 61

Глава 7. CODE REVIEW 68

Глава 8. ТВОРЧЕСКАЯ РАБОТА 73

Глава 9. ПРАКТИЧЕСКИЕ РАБОТЫ 80
9.1. Практическая работа № 1. Система контроля версий Git 80
9.2. Практическая работа № 2. CI/CD 86
9.3. Практическая работа № 3. Руководство по стилю и code review 87
9.4. Практическая работа № 4. TDD 90
9.5. Практическая работа № 5. Рефакторинг кода 91
9.6. Практическая работа № 6. Архитектуры и паттерны 93

Список литературы 93

Предисловие

Основа современных процессов разработки информационных систем – разработка и эксплуатация программного обеспечения. Увеличение объемов требований к различным навыкам программистов и процессы непрерывной интеграции программного обеспечения требуют участия команды различных, а иногда и территориально распределенных специалистов по разработке, тестированию и эксплуатации как программного обеспечения в общем, так и отдельных его частей. Все больше играет роль специализация работника. При этом программное обеспечение должно оставаться целостным, а процесс разработки – согласованным.

Коллективная разработка такого сложного программного обеспечения требует знаний процессов управления коллективом и процессов разработки. В процессе обучения студент выполняет работы в одиночку с четко поставленным техническим заданием и сроками исполнения, однако опыта и понимания процессов коллективной разработки на практике он не приобретает. В связи с этим особое внимание в пособии уделяется обязательной командной разработке большого проекта-задания, в котором несколько студентов коллективно разрабатывают информационную систему наподобие стартапа.

Командное взаимодействие студентов и преподавателя наиболее важно для коллективной разработки, поэтому большое внимание в учебном пособии уделяется методологиям разработки программного обеспечения, и в том числе – гибким методологиям, в которых закладывается возможность изменения технического задания. В случае студенческих работ это позволяет модифицировать задание в процессе его выполнения.

Учебное пособие также знакомит студентов с технологиями, применяемыми в процессе коллективной работы над информационными системами и программным обеспечением. Для работы с версиями программного обеспечения применяются системы контроля версий, и особое внимание уделяется системе контроля версий Git.

Для знакомства студентов с процессами развертывания программного обеспечения и реализации частой поставки версий изучается CI/CD. CI (continuous integration) применяется для непрерывной интеграции написанного модуля, программного кода в готовое приложение, а CD (continuous delivery или continuous deployment) позволяет непрерывно поставлять разрабатываемый код на «боевой» (production) сервер для непрерывной поставки обновлений пользователю.

Знакомство с тестированием программного обеспечения осуществляется через TDD (test-driven development).

Командная разработка включает и постоянное взаимодействие студентов, и общую ответственность за код. Для этого изучаются процессы code review и рефакторинга, нацеленные на улучшение и переработку кода. В результате студенты могут учиться у друг друга на готовом программном обеспечении, а общее владение кодом позволяет продолжить разработку сложного программного проекта и после завершения курса частью команды.

Архитектурные особенности разработки программного обеспечения в данном пособии рассматриваются в виде подхода к декомпозиции программы, особенность которого заключается в простоте его реализации на небольших программах. При командной студенческой разработке такой подход позволяет студентам проникнуться правильным описанием процессов декомпозиции.

В связи с тем, что паттерны проектирования строятся на знании концепции объектно-ориентированного подхода, который может быть не знаком некоторым студентам, паттерны проектирования и подходы к созданию архитектуры: микросервисы, CleanArchitecture и т. д. – становятся вторичными при начальном изучении основ создания архитектуры программного обеспечения.

Глава 1. МЕТОДОЛОГИИ УПРАВЛЕНИЯ ПРОЦЕССОМ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1.1. Водопадная модель (waterfall model)

Водопадная модель является классической моделью разработки программного обеспечения, наиболее часто применяемой как в отдельных проектах, так и студентами. В данной модели последовательно выполняются все этапы жизненного цикла: определение требований, анализ, проектирование архитектуры, написание программного кода, тестирование, внедрение продукта, сопровождение и утилизация. Модель получила свое название из-за своего линейного характера, при котором каждый этап процесса следует за предыдущим (напоминает падение воды).

После успешного завершения тестирования происходит этап установки и внедрения, на котором программное обеспечение устанавливается на целевой системе и запускается.

Основным преимуществом водопадной модели является ее простота и понятность. Каждый этап процесса четко определен и последовательно выполняется. Однако недостатком водопадной модели является ее жесткость и неспособность к адаптации к изменениям. Если требования меняются по ходу процесса разработки, то это может привести к серьезным проблемам.

В целом водопадная модель остается одной из самых распространенных методологий разработки программного обеспечения благодаря своей простоте и понятности. Она подходит для небольших проектов с четко определенными требованиями, где изменения маловероятны. Хорошим примером являются студенческие лабораторные работы: требования к лабораторным работам жестко зафиксированы, часто зафиксированы и способы реализации программного обеспечения. Студент осуществляет реализацию программного продукта, попутно обеспечивая его отладку. Тестирование программного обеспечения осуществляет преподаватель, в процессе тестирования может возникнуть необходимость доработки программного обеспечения, но требования при этом не изменяются.

Отдельно следует отметить процесс отладки. Отладка (debugging) – это процесс нахождения и исправления ошибок в программном коде. Она является неотъемлемой частью процесса разработки программного обеспечения и позволяет обнаружить и устранить проблемы, которые могут возникнуть во время выполнения программы. Целью отладки является достижение корректной работы программы без ошибок. Отладка помогает улучшить качество программного продукта и обеспечить его стабильную работу в различных условиях. Отладка может быть проведе-

на с использованием различных инструментов, таких как отладчики (debuggers), которые позволяют шаг за шагом выполнять код и отслеживать состояние программы. В процессе отладки разработчики могут использовать различные методы, например, добавление отладочных выводов в код, использование точек останова (breakpoints) и анализ стека вызовов (call stack).

Подобный подход не предполагает постоянного взаимодействия с пользователями или заказчиками, что затрудняет разработку, особенно на поздних этапах. Далее рассматриваются подходы, учитывающие изменяющиеся требования заказчика.

1.2. Итерационная и спиральная модели

К причинам изменения требований относят изменение рынка, желание заказчика или доработку программного обеспечения. Такое изменение возникает чаще всего при наличии некоторого варианта программного обеспечения, обладающего неполной функциональностью.

Итерационная модель разработки программного обеспечения – это методология, основанная на последовательном выпуске версий продукта через серию итераций, или циклов, разработки. Каждая итерация – это небольшой цикл разработки, в рамках которого выполняются все этапы разработки от начального проектирования до тестирования и внедрения. Итерационная модель предполагает возможный переход на любую предыдущую стадию разработки.

Когда программное обеспечение лучше описывает требования заказчика и бизнеса, достоинства такой разработки достаточно просты и очевидны, но его недостатки требуют отдельного пояснения. Для итерационной модели возможны множественные переходы, возвращения к предыдущим этапам. Требуется четко определить правила возврата к предыдущей стадии, например, что должно произойти, чтобы на этапе тестирования было необходимо провести повторный этап разработки или этап анализа. Установка таких критериев – трудоемкая и ответственная задача.

Основные принципы итерационной модели:

- Постепенное улучшение продукта. Программное обеспечение разрабатывается постепенно, и с каждой итерацией добавляются новые функции, улучшения и исправления.
- Частая обратная связь. Обратная связь – это влияние на разработку заказчика и пользователей, которое приводит к постоянной модернизации планов разработки с целью модификации программного обеспечения под меняющиеся потребности бизнеса.

- Повторное использование кода. Каждая итерация может использовать уже написанный и протестированный код, что способствует повышению эффективности разработки.
- Постоянное тестирование. Тестирование проводится на каждом этапе разработки, что позволяет выявлять проблемы и ошибки на ранних стадиях.

Спиральная модель разработки программного обеспечения – это методология, которая объединяет элементы итерационной и инкрементальной моделей разработки, а также учитывает особенности рискового управления.

Спиральная модель представляет собой циклический процесс, который включающий несколько оборотов, каждый из которых состоит из одной итерации.

Итерационный и спиральный подходы требуют от студента умения дорабатывать программное обеспечение под изменение требований преподавателя-заказчика. Такие требования призваны показать студенту цикличность жизненного цикла программного обеспечения, возможности его улучшения и доработки функциональности.

Трудности применения подобного подхода возникают из-за простоты разрабатываемого студентом программного обеспечения, ограниченности во времени и технологиях. Разработчик пытается провести полный анализ требований, определить всю структуру базы данных, архитектуру программного обеспечения, методы взаимодействия. Эти требования затягивают разработку на ранних этапах. Для ускорения итерации при разработке применяются функции-заглушки, которые позволяют эмулировать часть функционала программы. Такие заглушки, оставшиеся на этапе сдачи программного обеспечения, являются узкими местами в безопасности программного обеспечения, но само их наличие говорит об итеративности процесса разработки.

Основные черты спиральной модели:

- Цикличность. Разработка продукта происходит в виде циклов, каждый из которых включает четыре основных этапа: определение целей, оценка рисков, разработка и тестирование, планирование следующей итерации.
- Ориентация на риски. Основной принцип спиральной модели – управление рисками. На каждом этапе разработки проводится анализ и оценка рисков, что позволяет принимать обоснованные решения и минимизировать вероятность возникновения проблем.
- Инкрементальность. Продукт разрабатывается постепенно, с каждой итерацией добавляются новые функции и улучшения. Это позволяет заказчику получить рабочий продукт уже на ранних этапах разработки.

- Гибкость и адаптивность. Спиральная модель позволяет быстро реагировать на изменения требований или обстоятельств благодаря цикличности процесса разработки.
- Обратная связь. Важным элементом спиральной модели является обратная связь от заказчика или пользователей на каждом этапе разработки. Это помогает уточнить требования, учесть пожелания заказчика и повысить удовлетворенность конечным продуктом.

1.3. Модель прототипирования

Спиральную модель и модель прототипирования часто путают. В спиральной модели программное обеспечение дорабатывается с дополнительными изменениями, но в отличие от версий программного обеспечения (в итеративном процессе) прототипы не требуют доработки. Прототип разрабатывается для апробирования идей, подходов, технологий, для согласования с заказчиком требований и интерфейсов.

Основные черты модели прототипирования:

- Создание прототипа. На начальных этапах разработки создается прототип продукта, который может быть как простым наброском интерфейса, так и функциональным прототипом с базовыми функциями. Прототип помогает заказчику и разработчикам лучше понять требования к продукту.
- Обратная связь. Прототип предоставляется заказчику для оценки и обратной связи. Заказчик может вносить коррективы, предлагать улучшения или изменения, что позволяет уточнить требования и избежать недопонимания.
- Итеративность. Процесс разработки по модели прототипирования может быть итеративным, т. е. после получения обратной связи и внесения изменений в прототип процесс повторяется до достижения удовлетворительного результата.
- Сокращение рисков. Создание прототипа позволяет быстро определиться с технологией, что помогает уменьшить риски неверного принятия решений на ранних этапах разработки программного обеспечения.
- Быстрая разработка. Модель прототипирования позволяет быстро создавать промежуточные версии продукта, что ускоряет процесс разработки и позволяет заказчику быстрее получить рабочий продукт.

1.4. V-образная модель разработки

В основе V-образной модели разработки лежит подход к формированию требований через тестирование. Основываясь на разработке через тестирование TDD, разработки через требования BDD- и GWT-подходов

можно формализовывать требования заказчика в виде наборов высокоуровневых тестов. Выполнение данных тестов гарантирует выполнение программой бизнес-требований заказчика.

На дальнейших этапах подготовки к разработке программного обеспечения определяются системные, интеграционные, функциональные и модульные тесты.

В дальнейшем процесс разработки программного обеспечения сводится к написанию программного обеспечения, выполняющего полученные тесты. Сопровождение и доработка обеспечивается написанием новых (или изменением старых) тестов и доработкой под новые тесты программного обеспечения.

- **Test-Driven Development** (TDD). TDD предполагает написание тестов перед написанием кода. Разработчик начинает с написания теста, который описывает желаемое поведение программы. Затем он пишет минимальный необходимый код, чтобы тест прошел успешно, и, наконец, происходит рефакторинг кода для улучшения его качества. Таким образом, тесты становятся спецификацией требований к программе.
- **Behavior-Driven Development** (BDD). BDD фокусируется на поведении системы и позволяет описывать требования в форме спецификаций поведения (или сценариев). В BDD используется специальный язык, такой как Gherkin, для написания сценариев, которые описывают, как должна вести себя программа в различных ситуациях.
- **Given-When-Then** (GWT). GWT – это структурированный способ описания сценариев в BDD. Он состоит из трех частей: Given (Дано), When (Когда) и Then (Тогда). В разработке через требования с использованием GWT формулируются конкретные условия (Given), действия (When) и ожидаемые результаты (Then), что четко определяет поведение программы.

1.5. Гибкие методологии разработки программного обеспечения

Гибкие методологии разработки программного обеспечения (agile methodologies) ориентируются на частую итеративность и ориентацию на постоянное взаимодействие с заказчиком. Гибкие методологии предполагают разработку программного обеспечения в коротких циклах, называемых итерациями, спринтами и т. д. Каждая итерация создает новый инкремент функциональности, который может быть протестирован и показан заказчику. Гибкие методологии акцентируют внимание на способности к адаптации, изменениям в процессе разработки. Команды могут быстро реагировать на новые требования или обстоятельства, пересматривая планы и приоритеты. Заказчик играет активную роль в процессе разработки, участвуя в планировании, обсуждении требований и приня-

тии решений. Команды, работающие по гибким методологиям, обладают коллективной ответственностью за результат проекта.

Dynamic Systems Development Method (DSDM) – методология разработки программного обеспечения, которая сосредоточена на быстрой и итеративной разработке проектов. DSDM была создана в Великобритании в конце 1990-х гг. и является одной из agile-методологий. DSDM может быть успешно применена в проектах с жесткими сроками и требованиями к быстрому выводу продукта на рынок. Она позволяет эффективно управлять изменениями, обеспечивать качество продукта и сокращать время разработки.

Microsoft Solutions Framework (MSF) – фреймворк разработки программного обеспечения, выпущенный компанией Microsoft. MSF представляет собой набор принципов, моделей и практик, помогающий организациям эффективно управлять процессом создания программного обеспечения, который может быть успешно применен в различных типах проектов, включая крупные корпоративные системы, веб-приложения, мобильные приложения и др. MSF помогает организациям снизить риски, повысить качество продукта и достичь поставленных целей проекта.

Adaptive Software Development (ASD) – методология разработки программного обеспечения, которая акцентирует внимание на адаптивности и гибкости процесса разработки. ASD была разработана Джимом Хайсмитом (Jim Highsmith) и представляет собой итеративный и инкрементальный подход к созданию ПО. ASD может быть эффективно применена в проектах, где требуется быстрая адаптация к изменениям, высокая степень гибкости и сотрудничества между участниками.

eXtreme Programming (XP). XP была разработана Кентом Беком (Kent Beck) и Роном Джеффрисом (Ron Jeffries) в конце 1990-х гг. и основывается на четырех ценностях: коммуникация, простота, обратная связь и смелость. eXtreme Programming широко применяется в проектах, где требуется высокая скорость разработки, частая обратная связь от заказчика и гибкость в реагировании на изменения. Методология XP помогает улучшить качество продукта, повысить производительность команды и достичь более высокой удовлетворенности клиентов.

Основные принципы XP:

1. Парное программирование.

Два разработчика работают вместе за одним компьютером. Один пишет код, а другой следит за качеством, предлагает улучшения и задает вопросы.

Пример. Разработчик А пишет код для функции, а разработчик В предлагает оптимизации и проверяет код на соответствие требованиям.

2. Тестирование на ранних этапах.

Написание тестов до написания кода помогает определить требования и ожидаемое поведение программы.

Пример. Перед написанием функции разработчик пишет тесты, которые должны проходить успешно после написания кода.

3. Интеграция происходит постоянно.

Код интегрируется в общий репозиторий несколько раз в день, чтобы избежать конфликтов и обеспечить актуальность кода.

Пример. После завершения разработки новой функциональности код сразу интегрируется в основную ветвь проекта.

4. Маленькие релизы.

Частые релизы позволяют получать обратную связь от заказчика, быстро реагировать на изменения и улучшать продукт.

Пример. Команда выпускает новую версию приложения каждую неделю, чтобы клиенты могли оценить новые функции и предложить улучшения.

5. Общий владелец кода.

Вся команда разработчиков несет ответственность за качество и результаты своей работы, что способствует сотрудничеству и эффективности.

Пример. Каждый член команды имеет доступ к всему коду проекта и может вносить изменения или исправления.

6. Простота.

Создание простого и минималистичного кода помогает легко поддерживать и модифицировать продукт в будущем.

Пример. Разработчики выбирают простые и эффективные решения для задач, чтобы избежать излишней сложности.

Kanban – это методология управления рабочим процессом, которая акцентирует внимание на визуализации работы, ограничении рабочего потока и постоянном улучшении процесса.

Основные принципы Kanban включают:

1. Визуализация работы. Kanban-доска является основным инструментом визуализации рабочего процесса. На доске задачи представлены в виде карточек, которые проходят различные этапы выполнения (например, «В ожидании», «В процессе», «Завершено»). Это позволяет всем участникам видеть текущее состояние работы и легко отслеживать прогресс.

2. Ограничение рабочего потока (WIP limit). Kanban предлагает установление ограничений на количество задач, которые могут быть одновременно в работе. Это помогает снизить перегрузку команды, улучшить концентрацию на выполнении задач и сократить время выполнения задач.

3. Управление потоком. Kanban поддерживает плавное движение задач через рабочий процесс, обеспечивая оптимальное распределение ресурсов и минимизацию времени простоя.

4. Постоянное улучшение процесса. Kanban поощряет команду к регулярному анализу и улучшению своего рабочего процесса. Это может включать внедрение новых практик, оптимизацию текущих процессов и поиск возможностей для повышения эффективности.

Команда разработки программного обеспечения использует kanban-доску для визуализации задач по разработке новых функций. Каждая задача представлена карточкой на доске, которая проходит этапы: «Ожидание код-ревью», «В процессе разработки», «Тестирование» и «Готово к релизу». Ограничение WIP limit на каждом этапе помогает контролировать рабочий поток и улучшить эффективность работы.

Scrum – гибкий метод управления проектами, который акцентирует внимание на итеративной и инкрементальной разработке, самоорганизации команды и постоянном обучении.

Основные принципы Scrum:

1. Итеративная и инкрементальная разработка: Scrum предлагает разделение проекта на короткие временные интервалы, называемые спринтами, обычно длительностью от 1 до 4 недель. Каждый спринт заканчивается созданием готового инкремента продукта. В результате заказчик может динамически видеть увеличение функционала продукта и модифицировать требования под задачи быстро изменяющегося рынка

2. Роли в команде. Scrum определяет три основные роли: Product Owner (владелец продукта), Scrum Master (мастер Scrum) и команда разработки. Product Owner отвечает за определение требований и приоритизацию задач, Scrum Master поддерживает команду в применении методологии Scrum, а команда разработки отвечает за выполнение задач.

3. События. Scrum определяет несколько обязательных событий, таких как Sprint Planning (планирование спринта), Daily Scrum (ежедневное совещание), Sprint Review (обзор спринта) и Sprint Retrospective (ретроспектива спринта), которые помогают команде организовать работу, синхронизироваться и улучшать процессы.

4. Продуктовая очередь. Product Owner поддерживает список задач, известный как продуктовая очередь, который содержит все требования к продукту. Задачи в продуктовой очереди приоритизируются по ценности для бизнеса.

DSDM (Dynamic Systems Development Method), XP (Extreme Programming), Scrum и Kanban – все они принадлежат к семейству agile-методологий, но имеют свои особенности и отличия:

1. DSDM – это методология, которая акцентирует внимание на доставке ценности для клиента и гибкости в адаптации к изменениям.

DSDM предлагает структурированный подход к итеративной и инкрементальной разработке, включая фазы, роли и практики, направленные на достижение успеха проекта.

Основные принципы DSDM включают активное участие заказчика, четкое определение ролей и обязанностей, постоянную коммуникацию и тесное сотрудничество между участниками проекта.

2. XP – это методология, которая акцентирует внимание на технические аспекты разработки программного обеспечения.

Основные практики XP включают в себя парное программирование, тестирование на уровне кода, непрерывную интеграцию, частые релизы и многие другие, чтобы обеспечить высокое качество кода и быструю реакцию на изменения.

XP также поддерживает принципы гибкости и коллективной работы, но с упором на технические аспекты разработки.

3. Scrum – это agile-методология, которая акцентирует внимание на управлении проектом через итерации, называемые спринтами.Основные роли в Scrum включают Scrum Master (ведущий проекта), Product Owner (владелец продукта) и команду разработчиков.

Scrum предлагает структурированные события, такие как ежедневные стендапы, планирование спринта, обзор спринта и ретроспективы, чтобы обеспечить прозрачность и эффективное управление проектом.

4. Kanban – это методология управления рабочим процессом, которая акцентирует внимание на визуализации работы, ограничении рабочего потока и постоянном улучшении процесса.

Основные принципы Kanban включают ограничение количества задач в работе (WIP limit), ясное определение этапов рабочего процесса и постоянные измерение и анализ метрик производительности.

Kanban не имеет фиксированных итераций, как Scrum, а предлагает более гибкий подход к управлению процессом разработки.

Глава 2. СИСТЕМЫ УПРАВЛЕНИЯ ПРОЕКТАМИ

Система управления проектами – это программное обеспечение, которое предоставляет различные инструменты для анализа и контроля хода работы для любого процесса, будь то расписание студента, бизнес-процессы предприятия или процессы разработки программного обеспечения. Она необходима для успешной реализации задач, связанных с выполнением определенных действий в течение заданного времени с целью создания нового продукта или услуги, а также достижения конкретных результатов. Эта система помогает организовать деятельность, учитывая все требования, такие как выполнение поставленных задач в установленные сроки и ограниченный бюджет.

Основными целями системы управления проектами является:

- Увеличение продуктивности работы сотрудников, концентрации над задачами и эффективности использования рабочего времени сотрудников.
- Улучшение эффективности работы проектного-менеджера в рамках управления проектом, задачами и персоналом.
- Увеличение эффективности и результативности работы компании в рамках управления отдельными проектами.

При управлении проектами ставятся следующие задачи:

- Предоставление всем сотрудникам необходимой информации посредством понятного и результативного инструмента.
- Поддержка процессов планирования и контроля исполнения.
- Оценка текущего состояния проекта и объемов выполняемых заданий.
- Предоставление мониторинговой системы для возможности анализа и управления процессами предприятия в реальном времени.
- Предоставление удобной и понятной системы взаимодействия с персоналом, выдачи заданий и командной разработки.
- Системы управления проектами распространены в фирмах и компаниях с большим количеством процессов и задач.

К положительным сторонам систем управления проектами можно отнести:

- Эффективное управление ресурсами.
- Возможности фильтрации и установки приоритетов различным задачам.
- Контроль сроков исполнения.
- Упрощение ведения процессов документооборота.

При этом в небольших самоорганизующихся группах применение систем управления проектами может быть малоэффективно из-за того, что требуется постоянная работа с системой: создание задач, разбиение

задач на составляющие, назначение ролей, отслеживание процессов и т. д. Эти действия требуют времени, и если такими задачами занимаются подчиненные, то резко падает эффективность их работы. Также стоит отметить, что усложнение процессов управления разработкой программного обеспечения, их формализация для небольших проектов может быть губительна.

Пример работы системы управления проектами

Рассмотрим облачную систему календарного планирования и контроля выполнения проектов Yougile. Для работы в Yougile создается доска, описывающая один проект. На ней могут быть размещены отдельные этапы, задачи с указанными дедлайнами, описаниями и чатами. В начале работы для kanban-доски создаются столбцы: «Сделать» (To Do), «Готовы к разработке», «Разработка завершена», «Тестирование», а каждый из представленных столбцов разбивается на два: Doing и Done. Также можно создавать столбцы-колонки в соответствии с этапами выполнения задачи. Определенные задачи располагаются в карточках, которые перемещаются между колонками в процессе выполнения работы над задачей-карточкой.

Возможна командная работа с общей доской в системе управления проектами. В таком случае создание карточек и управление работой над ними является отдельной задачей и в больших группах может выполняться отдельным лицом, например, в методологии Scrum такие задачи выполняет Product Owner. В такой системе количество карточек может быть очень велико. Разнородные карточки затрагивают не только процессы разработки программного обеспечения, но и управленческие задачи, собрания, различные состояния и сроки. При этом в системах управления проектами вводится маркировка карточек-заданий. Так, в системе управления Yougile имеется возможность указать исполнителей карточки, поставить для карточки метку. Для удобства управления каждая карточка может быть снабжена описанием, а обсуждения процесса выполнения карточки может проводиться в специальном чате. Работник может концентрироваться только на назначенных ему карточках, отражающих все рабочие процессы. При активном использовании системы управление задачами становится сложным процессом и требует хорошей концентрации работников.

Отображение текущего состояния задачи, процесса ее выполнения или состояния завершения задачи является ответственностью работника. Если забыть отметить задачи на день, процесс их выполнения и завершения, то взаимодействие с системой управления проектами будет неэффективным. Для решения данной проблемы для карточки должны устанавливаться сроки исполнения, дедлайны, зависимости и напоминания, например уведомления на почту. При этом система управления проектами Yougile осуществляет календарное планирование и отображает задачи

не только на kanban-доске, но и в ежедневнике-планировщике, где задачи расставлены по времени и удобно расположены для работника, а также имеются диаграммы Ганта для удобства планирования ресурсов и последовательностей задач.

Уменьшение влияния человеческого фактора, например состояний «я не смотрел», «я забыл» и т. д., возможно при активной работе проектного менеджера (project-manager), который взаимодействует с работниками с целью обеспечения правильных процессов разработки и управления командой. Проектный менеджер уточняет у работника данные о его взаимодействии с системой управления проектами, напоминает о необходимости перемещений и завершений задач и т. д.

При использовании системы управления Yougile рекомендуется сделать ряд упражнений для ознакомления с широкими, постоянно обновляющимися возможностями системы управления проектами с целью изучения. Предлагается проделать следующие действия:

1. Установить программу Yougile, скачав ее с официального сайта https://ru.yougile.com/.
2. Создать компанию и пригласить туда друга для исследования возможностей совместной работы.
3. В компании создать доску и два проекта, назвав их разными именами. В первом проекте друг будет участвовать, во втором – нет.
4. Добавить для доски стикеры с набором состояний, задать три состояния.
5. Создать задачу без дедлайна.
6. Создать задачу с установленным дедлайном в виде конкретной даты и диапазоном дат.
7. Выбрать исполнителей для задачи.
8. Перетащить задачу между столбцами и отметить ее как выполненную.
9. Попросить друга создать новую задачу и назначить ее вам. Описать в чате данной задачи, что нужно сделать. Ответить, что приняли задачу к исполнению в чате.
10. Ознакомиться с возможностью отображения и перемещения задач, изменения длительности задач на диаграмме Ганта.
11. Ознакомиться с возможностью установки времени выполнения задачи и их отображения на виде календаря.
12. Создать регулярную задачу, поместив туда занятие. Попробовать включать и отключать регулярные задачи.
13. Дополнительно создать правило, позволяющее быстро управлять задачей, например, назначением зачета/экзамена и установкой новой задачи в виде проставления оценок в ведомость.

Глава 3. СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ

3.1. Понятие и виды систем контроля версий

Самая простая система контроля версий основана на хранении различных версий файлов целиком. Такое хранение может осуществляться студентом и без изучения систем контроля версий, путем создания файлов и папок виде ver1, с указанием даты и добавлением различных символов и цифр к имени старого файла. Такой подход неэффективен при частых версиях файлов и способствует созданию небольшого количества версий из-за необходимости хранения полных копий. При хранении только изменений в файлах возможно сокращение объемов файлов (хранится файл и изменения), но при этом требуется создание специального программного обеспечения для осуществления автоматического создания файла изменений и получения предыдущих версий.

Одним из важных понятий системы контроля версий является понятие репозитория. Репозиторий (repository) – это место, где хранится и управляется версионированным кодом, файлами, документацией и другими ресурсами проекта. Различные версии файлов хранятся в базе данных. В зависимости от места базы данных с изменениями системы контроля версий можно разделить: на локальные системы контроля версий (например, RCS), в которых база данных находится на локальном компьютере или рабочей станции, а в рабочую папку выбирается определенная версия кода из базы данных (рис. 3.1), и централизованные системы контроля версий (например, CVS, Subversion и Perforce), в которых база данных хранится на отдельном удаленным сервере (Remote Server), а рабочая папка находится у различных разработчиков на удаленных компьютерах (рис. 3.2).

Рис. 3.1. Локальная система контроля версий

При этом каждый разработчик может работать в своей версии программного обеспечения, перед началом работы необходимо обновить свою версию, а по завершении работы – загрузить версию программного обеспечения на удаленный репозиторий. Децентрализованные системы контроля версий (например, Git, Mercurial, Bazaar, Darcs) содержат базу данных с версиями программного обеспечения на каждом компьютере (рис. 3.3).

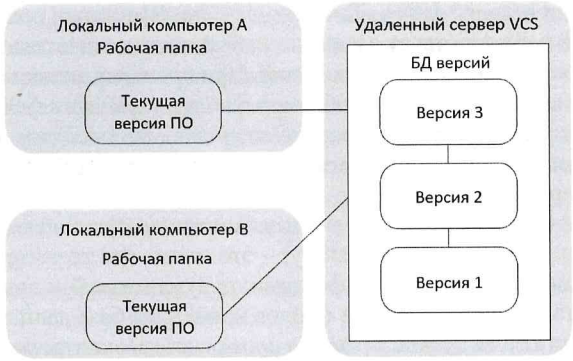


Рис. 3.2. Централизованная система контроля версий

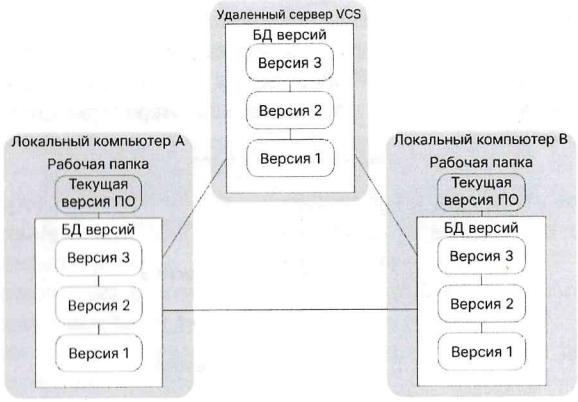


Рис. 3.3. Децентрализованная система контроля версий

Такой подход позволяет обновлять не рабочую папку, а базу данных версий и уже на основе выбранной версии формировать файлы в рабочей папке. Это обеспечивает полную независимость и безопасность данных, а также возможность работы с несколькими удаленными репозиториями

одновременно. Если удаленная связь с другими репозиториями отсутствует, то данная система фактически превращается в локальную систему контроля версий.

Хотя студенты чаще всего используют систему контроля версий Git и популярные платформы GitHub или GitLab для хостинга репозиториев Git, для общего представления о системах контроля версий приведем наиболее распространенные и используемые в прошлом системы.

Для представления основ работы с системами контроля версий приведены их особенности и небольшие фрагменты команд.

3.2. Системы контроля версий. Примеры

RCS (Revision Control System, www.gnu.org/software/rcs/rcs.html) – это одна из первых систем управления версиями, разработанная в 1981 г. Вальтером Францем Тихи (Walter F. Tichy) из Университета Пердью. Хранение текстовой информации происходит путем сохранения не полных копий файла, а первоначального файла и изменений, внесенных пользователем. Для каждого отдельного файла создается своя история хранения, своя база данных в отдельном файле, к имени которого добавляются символы «,v». Каждая версия бинарного файла записывается в базу данных в полном объеме. При работе с системой контроля версии различными пользователями применяется подход, подобный хранению и контролю версий программного обеспечения, записанного на перфокартах: файл, как и стопка перфокарт, полностью блокируются для изменений, когда над ним работает один пользователь. Встраивание RCS в современные операционные системы позволяет использовать подобные механизмы при нажатии комбинации CTRL+Z.

В RCS две основные команды ci (от check-in, регистрировать, **$ ci file.txt**) и co (от check-out, контролировать, **$ co file.txt**). Первая команда заносит изменения (если файл указан первый раз, то создает локальный репозиторий) в репозиторий, а вторая – загружает версию файла из репозитория.

CVS (Concurrent Versions System) – система управления параллельными версиями (www.nongnu.org/cvs), разработанная Дик Грун (Dick Grune) в середине 1990-х гг. Хранение файлов на локальном компьютере в системе контроля версий CVS выполняется аналогично системе контроля версий RCS, но при этом есть возможность хранения изменений не только для отдельных файлов, но и для директорий/папок. CVS является централизованной системой контроля версий и хранит все изменения на удаленном репозитории. После 2000-х гг. (последняя версия 2008 г.) считается устаревшей и не поддерживается.

Так как система контроля версий является распределенной, то и сохранение изменений осуществляется в виде отдельных слепков файлов, называемых коммитами (commit). Работа с удаленным сервером и воз-

можность хранения версий директорий изменили подход к командам системы контроля версий. Командой **$ cvs import** будет создана система контроля версий для текущей директории с указанием места хранения удаленного репозитория. Команда **$ cvs checkout** выполняется аналогично со и получает данные из удаленного репозитория, а команда **$ cvs commit** аналогично ci отправляет изменения на сервер.

Система управления версиями Subversion (SVN) (www.subversion.tigris.org) была разработана в 2004 г. по инициативе фирмы CollabNet. С 2010 г. Subversion является одним из проектов Apache Software Foundation и официально называется Apache Subversion.

По данным статистики использования пакетов Linux-дистрибутивов, количество активных пользователей Subversion достигло максимума примерно в 2010 г. и начало снижаться с 2016 г. Тем не менее количество проектов, использующих Subversion, все еще больше, чем использующих CVS, Mercurial и Bazaar.

Команды **Subversion** фактически повторяют команды CVS, но правила работы удаленного репозитория отличаются. Каждая команда к удаленному репозиторию считается транзакцией, имеется возможность работы с ревизиями, аналогом ветвей, поддерживаются (хотя и достаточно слабо) механизмы слияния ветвей проекта.

Также одним из недостатков проекта считается слабая поддержка переименования или удаления файлов. Это связано с тем, что в результате переименования файлов в системе контроля версий будет создан новый файл со своей историей и вернуться на предыдущую версию файла будет затруднительно.

Система управления версиями Aegis (www.aegis.sourceforge.net), созданная Питером Миллером в 1991 г., стала первой альтернативой централизованным системам управления версиями. Она основана на транзакциях и предоставляет основу для действий разработчиков над изменениями независимо друг от друга, а также координирует интеграцию этих изменений обратно в основной исходный код. Все операции в ней производятся через файловую систему Unix. Aegis поддерживает географически распределенное развитие, работу с несколькими репозиториями, наборами изменений, линиями разработки, одновременными активными ветвями и ветвление на любую глубину. Программа требует, чтобы наборы изменений работали, успешно строились и проходили тесты перед их интеграцией в базовый уровень проекта. Это связано с методологией экстремального программирования, активно влияющей на разработчиков системы контроля версий. Основной проблемой использования системы контроля версий Aegis является ее переусложненность, например, разработчики имеют доступ только к своим файлам, а сборка всего проекта происходит отдельно, децентрализованные репозитории синхронизируют изменения, но не передают все файлы программы для всех участников.

Пример работы с системой управления версиями Aegis.

Создание нового проекта:

$ aeproject create myproject

Добавление файлов в репозиторий:

$ aeadd file1.txt

Проверка статуса изменений:

$ aestatus

Зафиксировать изменения:

$ aeci -m "Добавлены файлы file1.txt и file2.txt"

Просмотр истории изменений:

$ aehistory

Работа с ветвями:

$ aebase -b newbranch

Слияние ветвей:

$ aemerge newbranch

Откат изменений:

$ aebreak -r 12345

Система управления версиями Monotone (monotone.ca). Monotone – децентрализованная система управления версиями, разработанная Грейдоном Хоэмом (разработка началась в 2003 г., релиз – в 2011 г.). В 2005 г. Линус Торвальдс рассматривал Monotone в качестве одного из вариантов замены BitKeeper, а впоследствии некоторые идеи Monotone были реализованы в Git.

Monotone написан на языке C++, распространяется в рамках лицензии GPL и позволяет организовать работу с набором обособленных репозиториев (каждый разработчик работает со своим локальным репозиторием). Объединить их в единое целое помогает эффективный P2P-протокол синхронизации. Для обеспечения надежности и целостности данных используются криптографические методы контроля (SHA-1 для хеширования изменений-ревизий, RSA для аутентификации и подписывания изменений-ревизий). Репозиторий с метаданными и информацией об изменениях хранится в SQLite-базе, отдельно от рабочего дерева исходных текстов.

Monotone поддерживает рабочий процесс diverge/merge, создание сетей с использованием пользовательского протокола netsync и импорт проектов CVS (система команд аналогична CVS).

Распределенность, безопасность и работа с системой контроля версий Monotone основана на transport- и commit-ключах. Transport-ключ дает вам возможность размещать ваши изменения в хранилище сервера Monotone. Без transport-ключа невозможно: получать код от сервера, который не предоставляет доступ для глобального чтения; размещать код на любом сервере; запускать сервер Monotone. Для фиксации кода в Monotone (по сути для подписания вашего кода) нужен commit-ключ. Без commit-ключа невозможно фиксировать любой код.

Monotone хранит ключи в $HOME/.monotone/keys в текстовых файлах, которые названы так же, как и ключи. Например: /home/complication /.monotone/keys/complication@mail.i2p

Для генерации transport- и commit-ключей необходимо выполнить следующие команды в командной строке:

$ mtn genkey yourname-transport@someplace

$ mtn genkey yourname@someplace

Monotone запрашивает пароль для защиты ваших ключей, поэтому рекомендуется установить пароль для commit-ключа. Большинство пользователей оставляют пустым пароль для transport-ключа, особенно те, кто запускает сервер Monotone.

Далее работа с сервером не представляет труда, например, основные команды работы с Monotone:

Инициализация репозитория:

$ mtn --db=repo.db db init

Добавление файлов в репозиторий:

$ mtn --db=repo.db add file.txt

Фиксация изменений (commit):

$ mtn --db=repo.db commit -m "Описание изменений"

Обновление локального репозитория из удаленного:

$ mtn --db=repo.db pull mtn://example.com/project

Отправка локальных изменений на сервер:

$ mtn --db=repo.db push mtn://example.com/project

Просмотр истории изменений:

$ mtn --db=repo.db log

Система управления версиями Mercurial (mercurial.selenic.com). Распределенная система контроля версий Mercurial разрабатывалась Мэттом Макалом в 2005 г. параллельно с системой контроля версий Git, созданной Торвальдсом Линусом. Большая часть системы контроля версий написана на языке Python, и только отдельные участки программы, требующие наибольшего быстродействия, написаны на языке Си и Rust. Mercurial использует консольную программу для работы, поддерживает полностью децентрализованную работу, ветвление и слияние репозиториев. Она также поддерживает обмен данными между репозиториями через HTTP/HTTPS, SSH и вручную с помощью упакованных наборов изменений. Mercurial имеет компактный интерфейс и считается более простой в освоении по сравнению с другими системами, такими как Git. Пример основных команд работы с системой управления версиями Mercurial (или hg):

Инициализация репозитория:

$ hg init

Добавление файлов в репозиторий:

$ hg add file.txt

Фиксация изменений (commit):

$ hg commit -m "Описание изменений"

Обновление локального репозитория из удаленного:

$ hg pull http://example.com/repo

Обновление локального репозитория и слияние изменений:

$ hg pull -u http://example.com/repo

Отправка локальных изменений на сервер:

$ hg push http://example.com/repo

Просмотр истории изменений:

$ hg log

Bazaar (bazaar.canonical.com) – распределенная, свободно распространяемая система контроля версий, разрабатываемая при поддержке компании Canonical Ltd. Основной особенностью системы контроля версий Bazaar является удобство работы с большим количеством ветвей и файлов, удобство слияния и графический интерфейс. Пример основных команд работы с системой управления версиями Bazaar (или bzr):

Инициализация репозитория:

$ bzr init-repo myproject

Создание ветви (branch) для разработки:

$ bzr branch myproject myproject-feature

Добавление файлов в репозиторий:

$ bzr add file.txt

Фиксация изменений (commit):

$ bzr commit -m "Описание изменений"

Обновление локального репозитория из удаленного:

$ bzr pull http://example.com/repo

Обновление локального репозитория и слияние изменений:

$ bzr merge http://example.com/repo

Отправка локальных изменений на сервер:

$ bzr push http://example.com/repo

Просмотр истории изменений:

$ bzr log

На рис. 3.4 приведена временная шкала разработки и применения различных систем контроля версий.

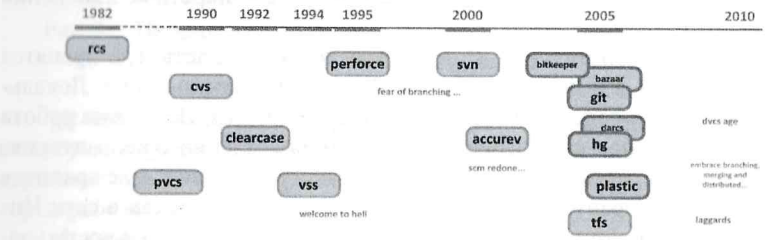


Рис. 3.4. Временная шкала разработки систем контроля версий

3.3. Система контроля версий Git

Git является одной из самых распространенных и тиражируемых систем контроля версий. Даже при условии, что разрабатывалась она для контроля версий сложных и больших технических проектов, в частности ядра ОС Linux, и требует умения взаимодействия с консолью, многие разработчики используют ее и для своих простых и небольших проектов. Во многом этому способствует широкое комьюнити и различные сервисы работы с системой контроля версий. Учитывая, что большинство систем контроля версий близки по принципам работы с ветвями и коммитами, а также близки по синтаксису и семантике команд, остановимся на системе контроля версий Git подробнее. В Git существует несколько основных областей, каждая из которых имеет свою роль и функциональность (рис. 3.5).

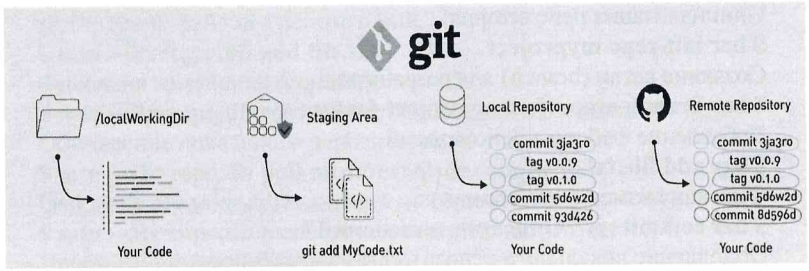


Рис. 3.5. Основные области системы контроля версий Git

Рабочая область (Working Directory) – место, где вы работаете над вашими файлами и вносите изменения. Все изменения, которые вы делаете в файлах, отображаются в рабочей области. Фактически это ваша папка с текущими версиями фалов, ваш код проекта.

Индекс (область индексации, Staging Area) – промежуточная область, где вы помещаете изменения, которые вы хотите включить в следующий коммит. Индекс позволяет вам выбирать конкретные изменения для коммита.

Локальный репозиторий (Local Repository) – область, где хранятся все версии файлов и история изменений на вашем компьютере. Локальный репозиторий содержит все коммиты, ветви и теги. Возможна работа только на локальном репозитории, без создания удаленного репозитория.

Удаленный репозиторий (Remote Repository) – область, где хранится копия вашего репозитория на удаленном сервере, чаще всего в сети Интернет. Таким образом обеспечивается безопасность и возможность удаленного взаимодействия с другими членами команды.

Временная область (Temporary Area) – область, где Git хранит временные данные при выполнении операций, таких как слияние или переключение ветвей.

Перенос изменений программного обеспечения, отдельных файлов, ветвей, коммитов и другой информации осуществляется с помощью команд, представленных на рис. 3.6.

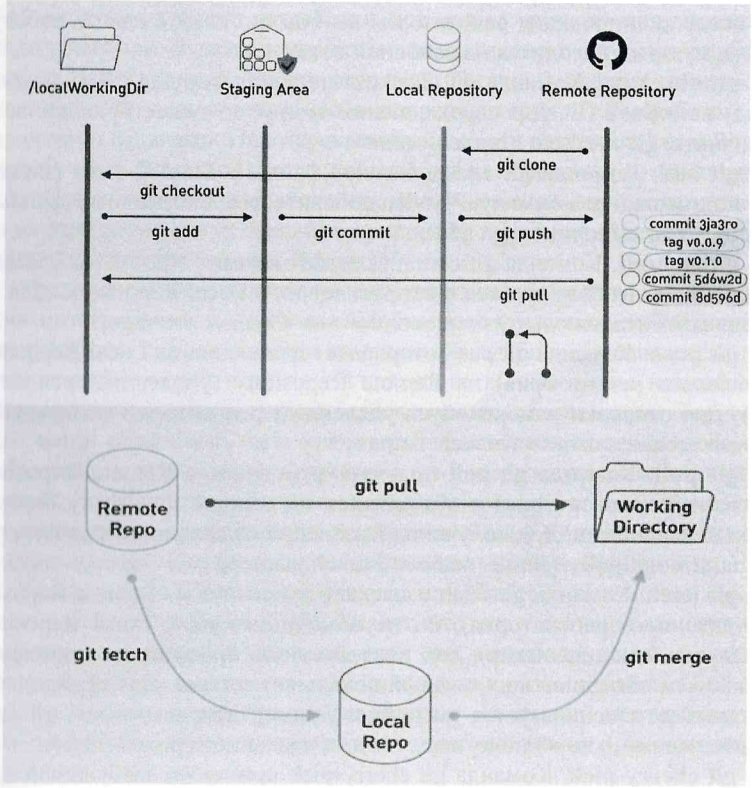


Рис. 3.6. Перенос изменений программного обеспечения из различных областей с помощью команд Git

git init. Создать пустой репозиторий Git или вновь инициализировать существующий. При инициализации создается скрытая папка, содержащая все объекты и ссылки, которые Git использует и создает в истории работы над проектом.

git clone. Команда git clone используется для клонирования удаленного репозитория Git на локальную машину. При клонировании репози-

тория с помощью git clone создается локальная копия удаленного репозитория со всей его историей и ветвями. Пример использования: git clone <repository_url>, где <repository_url> – ссылка на удаленный репозиторий.

После выполнения команды git clone Git автоматически переключает вас на основную ветвь (обычно это ветвь master), чтобы вы могли начать работу с клонированным репозиторием. Это происходит потому, что после клонирования репозитория вы обычно хотите начать работу с кодом, который находится на основной ветви.

git checkout. Команда git checkout используется для переключения между ветвями в Git. Для переключения на другую существующую ветвь выполните: git checkout <branch_name>.

git add. Команда git add добавляет файлы в Staging Area (индекс) для подготовки к коммиту. Чтобы добавить все измененные файлы в Staging Area, выполните: git add .

git commit. Команда git commit создает коммит с подготовленными изменениями из Staging Area и отправляет их в Local Repository. Для создания коммита используйте: git commit -m "Commit message".

git push. Команда git push отправляет изменения из Local Repository (локального репозитория) на Remote Repository (удаленный репозиторий). Для отправки изменений на удаленный репозиторий используйте: git push <remote_name> <branch_name>.

git pull. Команда git pull получает изменения с Remote Repository (удаленного репозитория) и **объединяет** их с Local Repository, локальными изменениями. Для получения изменений с удаленного репозитория выполните: git pull <remote_name> <branch_name>.

git fetch. Команда git fetch извлекает изменения из Remote Repository (удаленного репозитория), но **не объединяет** их с Local Repository (локальными изменениями). Это позволяет вам просмотреть изменения, прежде чем объединить их с вашей локальной ветвью. Для объединения в дальнейшем используется git merge. Пример использования: git fetch <remote_name>, где <remote_name> – имя удаленного репозитория.

git cherry-pick. Команда git cherry-pick применяет выбранный коммит к текущей ветви. Чтобы применить коммит из другой ветви, выполните git cherry-pick <commit_id>.

git rebase, git reset, git stash. Команды git rebase, git reset и git stash используются для перебазирования коммитов, отмены изменений и временного сохранения изменений соответственно.

Ветвь (Branch) определяет последовательность изменений, и версии сохраняются для отдельных ветвей. Первоначально в созданном репозитории имеется одна ветвь master или main. Если все изменения множество разработчиков добавляют в первоначальную ветвь, то перед каждым добавлением необходимо получить все другие измененные файлы из

данной ветви, что может привести к неработоспособности кода, как будто все программисты одновременно пишут код в одном проекте. Если все разработчики добавляют изменения в одну и ту же основную ветвь, это может привести к конфликтам и неработоспособности кода, поскольку все изменения могут пересекаться.

Для того чтобы избежать подобных проблем и разделить разработку на части, создаются новые ветви разработки. Каждая ветвь представляет собой отдельную линию разработки, где можно работать над определенной задачей или функциональностью параллельно с другими разработчиками. Переключение между ветвями (Checkout) позволяет переключаться между разными ветвями вашего репозитория, чтобы работать над различными задачами. При переключении между ветвями в Local Repository весь код в Working Directory обновится на последний коммит данной ветви. После завершения работы над задачей в отдельной ветви изменения из этой ветви могут быть интегрированы обратно в основную ветвь с помощью операции слияния (Merge).

Процесс слияния является ответственным, поскольку код из одной ветви интегрируется в другую и необходимо убедиться, что изменения интегрируются корректно. При командной разработке правила использования ветвей регламентированы и каждая ветвь имеет свое назначение с точки зрения хранения версий программного обеспечения. Так, чаще всего ветви master или main хранят рабочие версии программного продукта, кода, который протестирован и не содержит промежуточных состояний. Часто применяют ветвь dev (Development), в которой хранится код для полного проведения тестирования. Такой код может содержать неиспользуемые модули, недотестированные участки и функции.

Конфликты в Git возникают, когда при попытке объединить изменения из разных ветвей Git обнаруживает, что эти изменения взаимно противоречат друг другу. Например, если два разработчика одновременно внесли изменения в одну и ту же строку кода в разных ветвях, Git не может автоматически решить, какую версию изменений использовать. В этом случае Git отмечает конфликт и требует ручного вмешательства разработчика для его разрешения.

Когда возникает конфликт, Git помечает файлы, в которых обнаружены конфликты, специальными маркерами, указывающими на начало и конец конфликта. Разработчику необходимо вручную просмотреть эти файлы, решить, какие изменения следует сохранить и какие отклонить, и удалить маркеры конфликта. После разрешения всех конфликтов разработчик должен добавить изменения в индекс (git add) и завершить операцию слияния (git merge) или переподтвердить слияние (git commit).

Разрешение конфликтов требует внимательности и аккуратности, поскольку неправильное разрешение конфликта может привести к ошибкам в коде или потере важных изменений. Поэтому важно внимательно

проанализировать изменения и убедиться, что после разрешения конфликтов код остается работоспособным. Хорошая практика при работе с конфликтами – регулярно обновлять свою локальную копию репозитория перед началом работы над изменениями и перед слиянием ветвей. Это поможет уменьшить вероятность возникновения конфликтов и сделает процесс слияния более плавным.

Рассмотрим отдельные команды Git.

Команда **git config** используется для настройки параметров конфигурации репозитория, пользователя или системы. Вот несколько вариантов применения git config:

Настройка имени пользователя и email:

git config --global UserName "YourName"

git config --global UserEmail "student@example.com"

Установка редактора по умолчанию: **git config --global core.editor "nano"**

Эта команда устанавливает текстовый редактор, который будет использоваться для написания коммит-сообщений.

Настройка цветового вывода: **git config --global color.ui auto**

Эта команда включает цветовое выделение в выводе Git для лучшей читаемости.

Установка шаблона игнорируемых файлов: **git config --global core.excludesfile ~/.gitignore_global**

Эта команда устанавливает путь к файлу, который содержит шаблоны для игнорирования файлов при работе с Git.

Настройка поведения слияний: **git config --global merge.tool vimdiff**

Эта команда устанавливает инструмент для разрешения конфликтов слияний. В данном случае используется vimdiff, но можно выбрать другой инструмент: KDiff3, Beyond Compare, P4Merge, Araxis Merge

Просмотр текущих настроек: **git config --list**

Команды git config могут быть использованы как для локальной, так и для глобальной конфигураций репозитория (с флагом --global). Настройки, установленные с флагом --global, будут применяться ко всем репозиториям на вашем компьютере.

Для обозначения Remote Repository (удаленного репозитория), с которого был склонирован локальный репозиторий, обычно используется термин origin. Когда вы клонируете репозиторий с удаленного сервера, Git автоматически создает ссылку на этот удаленный репозиторий под именем origin. Ссылка origin на Remote Repository позволяет вам взаимодействовать с удаленным репозиторием, отправлять и получать изменения, а также управлять ветвями и слияниями. Например, когда вы выполняете команду git push origin master, вы отправляете изменения из локальной ветви master в Remote Repository origin. А команда git pull origin master позволяет вам получить изменения из удаленной ветви master и объединить их с вашей локальной ветвью. Если необходимо добавить изменения, но не создавать новый коммит, то опция --amend в Git позволяет внести изменения в последний коммит. Когда вы используете git commit --amend, Git добавляет изменения к предыдущему коммиту, а также позволяет вам отредактировать сообщение коммита. Важно помнить, что использование --amend изменяет историю коммитов, поэтому не следует использовать эту опцию для изменения коммитов, которые уже были опубликованы в Remote Repository (удаленном репозитории). В противном случае это может привести к проблемам синхронизации истории коммитов между Local Repository и Remote Repository.

Также полезными являются команды:

git status – показать состояние репозитория (отслеживаемые, измененные, новые файлы и пр.).

git diff – сравнить Working Directory и Staging Area.

git reset – убрать из Staging Area все добавленные в него изменения (в Working Directory все изменения сохранятся), антипод git add.

git clean –df – удалить неотслеживаемые файлы и директории.

git revert HEAD --no-edit – создать новый коммит, отменяющий изменения последнего коммита без запуска редактора сообщения.

git rm ind* – удалить все отслеживаемые файлы с именем, начинающимся на «ind» в текущей директории и проиндексировать это изменение.

git mv text.txt test_new.txt – переименовать файл «text.txt» в «test_new.txt» и проиндексировать это изменение.

git log master – показать коммиты в указанной ветви.

git show HEAD~ – показать данные о предыдущем коммите в активной ветви.

git blame README.md --date=short -L 5,8 – показать строки 5–8 указанного файла и коммиты, в которых строки были добавлены.

Отдельно стоит упомянуть облачные платформы хостинга проектов, например, GitHub, GitLab и т. д. Для децентрализованной работы систем контроля версий необходимо указать адреса всех участников проекта. Если все участники находятся в рамках одной сети, то данное действие не составляет труда, но в случае распределенной разработки и обмена версиями проекта через сеть Интернет потребуется установка внешних, маршрутизируемых в сети IP-адресов для каждого участника. Для решения данной проблемы многие разработчики отказываются от полной децентрализации проекта и используют централизованный облачный сервер (например, рис. 3.3), с возможностью хранения всей базы данных изменений на локальном компьютере. При этом в студенческих проектах и для студенческих работ такая платформа хостинга превращается в облачное хранилище программного обеспечения и не применяется как система контроля версий программного кода.

Глава 4. АРХИТЕКТУРЫ

4.1. Основные принципы создания архитектур. Универсальные критерии правильной архитектуры

При разработке программного обеспечения разработчик проводит декомпозицию кода, разделяя его на функциональные блоки, объекты и другие элементы. Этот процесс приводит к формированию архитектуры программного обеспечения, которая описывается в виде правил, стандартов и документации. Даже у новичков в программировании можно выделить архитектурные аспекты в их первых программах. Для выбора оптимальной архитектуры необходимо понять характеристики успешных и неудачных архитектур, а также принципы различных архитектурных шаблонов. Понимание этих факторов поможет создать эффективное и устойчивое программное обеспечение.

В учебном пособии мы рассматриваем основные принципы создания архитектур. Правила разработки конкретных архитектур рекомендуется изучать по специализированной литературе [1; 4; 7–9; 11; 13; 15; 17–19].

Общепринятого определения архитектуры программного обеспечения нет, но в процессе работы над программным обеспечением для большинства разработчиков понятно, какой код является хорошим, а какой – нет. Программисты создают выгодную для себя архитектуру, делающую процесс работы над кодом простым и эффективным.

Сформулируем универсальные критерии хорошей/правильной архитектуры:

Эффективность системы. Наверное, самый понятный критерий. Его формулировка достаточно проста: система должна выполнять требуемые от нее функции.

Гибкость системы определяет способность программного кода приспосабливаться к изменениям технического задания, добавлению нового функционала с минимальными правками. В гибкую систему изменения и дополнения вносятся быстро, разработчики работают с максимальной эффективностью.

Важными аспектами гибкости системы являются:

– легкость внесения изменений;

– минимизация влияния изменений на другие части системы;

– ограничение последствий архитектурных ошибок: хорошая архитектура должна предусматривать возможность допущения ошибок и обеспечивать их минимальное влияние на всю систему;

– отложенное принятие ключевых решений: хорошая архитектура позволяет откладывать принятие ключевых решений до момента, когда это действительно необходимо, чтобы избежать преждевременной оптимизации или неправильных выводов (Б. Мартин).

Гибкость архитектуры программного обеспечения облегчает обновление и поддержку программного кода.

Расширяемость системы. Правильная архитектура программного обеспечения позволяет добавлять функциональность, новые сущности без необходимости изменения структуры программы. На начальном этапе разработки программы принцип YAGNI (you ain't gonna need it, «Вам это не понадобится») требует реализовывать только самый необходимый функционал, избегать излишней сложности и ограниченности требований и кода. Тем не менее хорошая архитектура предполагает, что впоследствии возможно добавление нового функционала, и требуется учитывать такие возможности при проектировании.

Масштабируемость процесса разработки включает в себя возможность эффективного увеличения размеров команды разработчиков и ускорения процесса создания программного продукта за счет добавления новых участников к проекту. Архитектура программного обеспечения должна обеспечивать возможность распараллеливания работы разработчиков таким образом, чтобы множество людей могли одновременно работать над различными частями программы.

Для обеспечения масштабируемости процесса разработки следует учитывать следующие аспекты:

- *Модульная структура.* Разделение программы на модули или компоненты позволяет разработчикам работать над отдельными частями программы независимо друг от друга.
- *Использование систем контроля версий.* Использование распределенных и децентрализированных систем контроля версий позволяет нескольким разработчикам одновременно работать над одним проектом, управлять изменениями и объединять свою работу без конфликтов.
- *Стандартизация кода и документации.* Установление стандартов для написания кода и документации помогает новым участникам команды быстрее встать на «рабочие рельсы» и начать активное участие в процессе разработки.
- *Использование инструментов для совместной работы.* Использование средств командной разработки, таких как системы управления задачами (например, YouGile), обмен документами (например, Yandex Диск) и коммуникации (например, Yandex Telemost), упрощает совместную работу разработчиков и обеспечивает эффективное взаимодействие в больших командах.
- *Автоматизация процессов.* Автоматизация тестирования, сборки и развертывания программы, процессы CI/CD помогают ускорить процесс разработки и облегчить интеграцию новых участников в команду.

Масштабируемость процесса разработки влияет на возможности легкого расширения команды, найма и обучения новых сотрудников с минимальными потерями в эффективности и работоспособности остальных членов команды.

Тестируемость кода. Тестирование не только помогает улучшить качество кода, но и оказывает значительное влияние на сам дизайн программы. Многие разработчики приходят к выводу о том, что требование хорошей тестируемости является направляющей силой, автоматически ведущей к хорошему дизайну. Чтобы код был легко тестируемым, он должен быть хорошо структурированным, модульным, с четкими интерфейсами и минимизированными зависимостями между компонентами. Принцип тестируемости кода можно использовать как своеобразную лакмусовую бумажку для оценки качества его дизайна. Даже если вы не пишете тестовый код, ответ на вопрос о том, насколько легко и эффективно можно будет протестировать код, может помочь понять, насколько хорошо или плохо он спроектирован.

Кроме того, тестируемость напрямую связана с возможностью дальнейшего сопровождения и расширения продукта. Хорошо тестируемый код облегчает выявление и исправление ошибок, уменьшает технический долг и позволяет быстрее вносить изменения или добавлять новые функциональности. В результате тестируемость кода не только способствует повышению его качества, но и улучшает общую производительность команды разработки.

Хорошо структурированный, читаемый и понятный код является ключевым аспектом **сопровождаемости программного продукта**. В разработке программы участвует множество людей, и после завершения проекта новые разработчики должны легко разобраться в коде. Применение стандартных и общепринятых решений в проекте способствует простому общению, быстрому изучению и легкому пониманию кода другими разработчиками.

Принцип наименьшего удивления (principle of least astonishment) также играет важную роль в сопровождении программы. Суть принципа очень проста: код, дизайн, UI, тесты и т. п. – должны быть логичны, последовательны, соответствовать общепринятым (среди пользователей/команды) практикам и в целом не удивлять негативно. Этот принцип подразумевает, что код должен быть написан таким образом, чтобы его поведение было интуитивно понятным для других разработчиков. Чем более стандартными и привычными будут решения в программе, тем легче будет в ней разбираться новым участникам команды. К тому же относится выбор языков программирования, систем управления базами данных, паттернов, архитектурных шаблонов, дополнительных сервисов и программного обеспечения.

4.2. Декомпозиция – основа архитектуры

В процессе разработки программы неизбежно расширяется ее кодовая база. Это увеличивает сложность программного обеспечения. Еще на этапе обучения студенты инстинктивно разделяют код на части, применяя в своих лабораторных работах функции и объекты, а в больших дипломных и курсовых проектах – модули. Модульной структурой также обладает большинство программных фреймворков. Такой подход к организации кода иногда называется принципом «разделяй и властвуй» (divide et impera), но в основе этого лежит **иерархическая декомпозиция**. При декомпозиции разработчики стремятся уменьшить сложность системы, выделить отдельные компоненты, начать повторно использовать код, но при этом получают и улучшение гибкости системы, хорошие возможности для масштабирования и увеличения устойчивости. Декомпозированный код проще тестировать, сопровождать и документировать.

Кроме выделения отдельных блоков (функциональных модулей, сервисов, слоев, подпрограмм) при декомпозиции особое внимание уделяется связям между данными блоками. Чем более независимыми будут эти блоки, тем безопаснее сосредоточиться на разработке каждого из них в отдельности в конкретный момент времени, не беспокоясь о других частях системы.

Можно утверждать, что основная цель всех методик проектирования заключается в разбиении сложной проблемы на более простые фрагменты. Термин «архитектура» чаще всего относится к результату этого разделения, а также к некоторым структурным решениям, которые считаются основополагающими и сложноизменяемыми. В большинстве определений архитектуры подчеркивается идентификация ключевых компонентов системы и способы их взаимодействия. Хорошая архитектура по сути представляет собой модульную или блочную структуру. Для создания качественной архитектуры необходимо уметь правильно декомпозировать систему. Это означает понимание того, какая декомпозиция считается правильной и как лучше проводить этот процесс.

4.3. Функциональная и иерархическая декомпозиции

Вопросы декомпозиции системы касаются не только программного обеспечения, но и множества других сложных структур. Из исследований данного вопроса программистами были взяты *подходы функциональной и иерархической декомпозиций*. Обычно такая декомпозиция осуществляется сверху вниз, т. е. сначала выделяются основные блоки, системы и подсистемы, а после этого подсистемы разбиваются на модули, классы и функции. Для небольших приложений часто достаточно пары уровней декомпозиции. В таких системах типичными модулями пер-

вого уровня, полученными после разделения системы на крупные части, могут быть бизнес-логика, пользовательский интерфейс, доступ к БД, связь с оборудованием или ОС. Аналогично с декомпозицией систем выделение на одном уровне более 7 модулей увеличивает сложность структуры. Для такой системы рекомендуется создавать промежуточные модули, уровни декомпозиции.

Функциональная декомпозиция подразумевает не только разделение методов, но и данных. При таком подходе важно, чтобы при декомпозиции происходило разделение по функциям. То есть каждый модуль должен решать определенную подзадачу и выполнять соответствующую функцию, причем иметь необходимые данные для этого. Такая декомпозиция позволяет разделить программное обеспечение на различные слабо связанные модули, легко масштабироваться и наращивать функциональность.

Модуль – это не произвольный кусок кода, а отдельная функционально осмысленная и законченная программная единица (подпрограмма), которая обеспечивает решение некоторой задачи и в идеале может работать самостоятельно или в другом окружении и быть переиспользуемой. Модуль должен быть некоей «целостностью, способной к относительной самостоятельности в поведении и развитии» (Кристофер Александер).

High Cohesion + Low Coupling. Непохожим на стандартные методы декомпозиции является декомпозиция по принципам High Cohesion и Low Coupling. Особенностью разработки программного обеспечения является наличие четко прописанных модулей и взаимосвязей. Можно провести декомпозицию, разделение на составляющие таким образом, чтобы минимизировать количество связей между модулями. Тогда при изменении внутренней структуры одного модуля проблемы в других модулях будут маловероятны и отладку необходимо проводить только в одном ограниченном модуле. При этом модули остаются сфокусированы на определенных функциях и задачах. Обычно такой подход формулируют следующим образом: модули, полученные в результате декомпозиции, должны быть максимально сопряжены внутри (high internal cohesion) и минимально связаны друг с другом (low external coupling).

- High Cohesion, высокая сопряженность, или сплоченность, внутри модуля. Все элементы, которые не имеют высокой сопряженности, должны быть вынесены в отдельные модули. Следствием High Cohesion является принцип единственной ответственности (Single Responsibility Principle – первый из пяти принципов SOLID), согласно которому любой объект/модуль должен иметь лишь одну обязанность и, соответственно, не должно быть больше одной причины для его изменения. Такой поход способствует созданию множества сильно связанных компонент и возможно превращения таких компонент в отдельные сервисы.

- Low Coupling, слабая связанность между модулями. Противоположное свойство, позволяющее разделить программный код на слабо связанные блоки. Они должны иметь возможность взаимодействовать, но при этом как можно меньше знать друг о друге (принцип минимального знания). Чем слабее связанность, тем легче писать (понимать/расширять/чинить) программу.

Считается, что хорошо спроектированные модули должны обладать следующими свойствами:

- *функциональная целостность и завершенность.* Каждый модуль должен быть завершенным, выполнять конкретные действия и в идеале быть самодостаточным;
- *один вход и один выход.* На входе программный модуль получает определенный набор исходных данных, выполняет содержательную обработку и возвращает один определенный набор данных, т. е. реализуется стандартный принцип IPO (In-Process-Out);
- *логическая независимость.* Модуль не вызывает и не обращается к другим модулям, на вход модуля только подаются запросы и данные;
- *слабые информационные связи с другими модулями.* Обмен информацией между модулями должен быть по возможности минимизирован.

Пределами такого подхода является полная автономия программных модулей, доведение их до сервисов. Подход с определением связанности и декомпозицией на сильные области может проводиться автоматическими или автоматизированными средствами. Возможна начальная разработка замкнутых, конечных, не зависящих друг от друга модулей.

4.4. Ослабление связанности между модулями. Decoupling

В основе архитектурных шаблонов и паттернов лежат принципы объектно-ориентированного программирования. Для дальнейшего изучения материала (далее будем предполагать наличие у обучающихся знаний в области объектно-ориентированного программирования) требуется иметь представление о паттернах проектирования «банды четырех», так как многие части архитектур и элементов взаимодействия являются различными структурными, поведенческими и порождающими паттернами. Для изучения паттернов рекомендуется изучение книги от четырех первоклассных разработчиков – «банды четырех» (Gang of Four, GoF)[1].

Главным, что позволяет уменьшать связанность системы, являются **интерфейсы** и стоящий за ними принцип «инкапсуляция + абстракция + полиморфизм». Инкапсуляция требует, чтобы не только объекты, но и

[1] Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб. : Питер, 2021.

модули и другие элементы не имели прямого доступа к внутренним элементам модулей. Для обмена информацией каждый модуль должен иметь набор абстракций, представленных интерфейсами и паттерном «фасад». Закрытый законченный модуль создать по принципам «черного ящика» можно относительно легко без влияния на остальную систему. Проблемы могут возникнуть только при взаимодействии различных модулей, изменении интерфейсов и фасадов. Для решения данной проблемы старые интерфейсы не изменяются, а добавляются новые. При использовании абстракций можно создавать контракты интерфейсов для возможности применения различных модификаций реализации данного интерфейса.

Полиморфизм с точки зрения архитектуры – это вовсе не переопределение методов, как иногда ошибочно полагают, а прежде всего взаимозаменяемость модулей/объектов с одинаковым интерфейсом, или «один интерфейс, множество реализаций». Механизма наследования следует избегать при реализации полиморфизма, а предпочтение отдать интерфейсам. Использование интерфейсов позволяет абстрагироваться от внутренней реализации подсистем и рассматривать их как единое целое, обеспечивая слабую связанность между модулями. Чем более общим и абстрактным является интерфейс, тем гибче становится система.

В области модульного проектирования интерфейс модуля может быть реализован с помощью специальной конструкции, известной как фасад. Фасад маскирует сложности и тонкости внутренней работы модуля, предлагая простой и удобный интерфейс для взаимодействия с другими компонентами системы. По сути фасад – это больше, чем просто объект интерфейса, скрывающий сложность и внутреннюю структуру подсистемы; он также функционирует как единая точка входа для взаимодействия с этой подсистемой. Используя фасад, мы можем управлять модулями так же, как и обычными объектами, используя стандартные принципы и методы проектирования классов.

Интересно, что хотя многие разработчики осознают важность интерфейсов объектного уровня, использование интерфейсов для уменьшения связи между модулями является относительно новой концепцией. Нечасто можно увидеть примеры использования фасадов при разработке архитектурных шаблонов, таких как MVC.

4.5. Замена прямых зависимостей

Переход от прямых зависимостей к синхронизации через централизованное (общее) ядро может существенно упростить взаимодействие системы. Этот метод основан на принципах шаблона проектирования «посредник». В системах с многочисленными модулями прямое взаимодействие между модулями может стать чрезвычайно сложным, по-

этому выгодно перейти от модели взаимодействия «многие ко многим» к модели «один ко многим». Это достигается за счет введения обобщенного посредника, которым может быть **общее ядро приложения**, централизованная система хранения или шина данных. Все остальные модули затем функционируют как независимые клиенты, используя услуги, предлагаемые этим ядром, или обрабатывая содержащиеся в нем данные.

Реализуя эту стратегию, клиентские модули могут взаимодействовать друг с другом косвенно через посредника, не зная о существовании друг друга. Централизованное ядро может принимать две основные формы: оно может либо знать клиентские модули и управлять их взаимодействием (аналогично архитектуре Apache), либо оно может оставаться в основном (если не полностью) независимым и не осведомленным о клиентах. Такая гибкость позволяет создавать масштабируемые и удобные в обслуживании конструкции систем, в которых модули разделены, что снижает сложность и повышает надежность системы.

По своей сути эта концепция представлена в шаблоне «**модель – представление – контроллер**» (MVC). В MVC несколько пользовательских интерфейсов (представлений) могут взаимодействовать с одной моделью (которая действует как центральный компонент приложения и общий репозиторий данных). Эти интерфейсы работают синхронно, не зная друг о друге, и модель не знает о них. Кроме того, он позволяет вспомогательным модулям, а не только интерфейсам, подключаться к общей модели для синхронизации.

Этот принцип также широко применяется при разработке игр. Независимые модули, отвечающие за графику, звук, физику и управление программой, взаимодействуют и синхронизируются через игровое ядро (модель), хранящее все данные о состоянии игры и ее персонажах. В отличие от MVC, при разработке игр синхронизация между модулями и ядром основана не на шаблоне «наблюдатель», а на времени. Это уникальное архитектурное решение особенно выгодно для приложений с анимацией и динамической графикой, поскольку оно обеспечивает гармоничное и последовательное управление различными аспектами игрового процесса.

Используя координацию на основе таймера, разработчики могут эффективно координировать действия различных модулей, поддерживая согласованность графических и физических компонентов игры. Такой подход обеспечивает более плавный и отзывчивый игровой процесс, когда изменения в одном модуле, такие как движение персонажа или физика окружающей среды, легко интегрируются с визуальными и звуковыми элементами, улучшая общее впечатление от игрока.

Использование обобщенного посредника в архитектуре приложений – один из факторов создания более гибких и масштабируемых си-

стем. Модули могут быть легко добавлены или изменены, оставляя незатронутыми другие части системы. Одним из ключевых преимуществ является уменьшение сложности взаимодействия между различными компонентами системы. Каждый модуль может работать независимо от других, при этом общая логика управления и координации выносится на обобщенный посредник.

Идея использования обобщенного посредника также способствует повышению безопасности системы. Поскольку модули не зависят напрямую друг от друга, это уменьшает возможность ошибок взаимодействия и повышает стабильность работы системы. Упрощается процесс разработки и поддержки приложений, так как каждый модуль может быть разработан и тестирован независимо от других, что ускоряет процесс разработки и повышает качество программного продукта. Посредник может быть использован для организации взаимодействия различных компонентов системы как в локальной среде, так и в распределенных системах.

Обобщенный посредник реализуется различными способами в зависимости от конкретных требований системы.

Ниже приведены некоторые из возможных способов реализации обобщенного посредника:

- Один из способов реализации обобщенного посредника – использование **шины данных (Data Bus)**, которая служит для передачи данных между различными компонентами системы. Шина данных может быть централизованной или децентрализованной в зависимости от архитектуры системы.
- Другой способ реализации обобщенного посредника – создание **центрального ядра приложения (Application Core),** которое управляет всеми компонентами системы и обеспечивает их взаимодействие. Ядро приложения может содержать логику управления, обработки данных и другие общие функции.

Сервис-ориентированная архитектура (Service-Oriented Architecture, SOA). В сервис-ориентированной архитектуре (SOA) каждый компонент системы рассматривается как отдельная служба, к которой другие компоненты могут получить доступ через обобщенного посредника, известного как сервисная шина или более конкретно **корпоративная сервисная шина (ESB, Enterprise Service Bus)**. Концепция ESB возникла, когда организации использовали несколько разрозненных приложений, например, одно для финансового управления, другое для учета персонала и третье для управления складом, причем все эти приложения необходимо было соединить между собой. Эти приложения, изначально разработанные без учета стратегии интеграции, не имели стандартного языка общения. Следовательно, разработчики создали конечные точки для отправки и получения данных в определен-

ных форматах. Затем компании-клиенты взяли на себя задачу интеграции этих приложений путем создания каналов связи и преобразования сообщений с языка одного приложения на другой. Эта архитектурная модель опирается на модульные приложения, часто ориентированные на пользователя, которые взаимодействуют с веб-сервисами для выполнения операций. Веб-службы, в свою очередь, могут взаимодействовать с другими веб-службами, в конечном итоге возвращая данные приложению. Примечательно, что ни приложения, ни серверные службы не обладают никакой информацией друг о друге, включая их местоположение и протоколы связи. Они знают только, к какой службе они хотят обратиться и где находится служебный автобус – шина данных. Когда клиент (служба или модульное приложение) отправляет запрос в служебную шину, шина преобразует сообщение в формат, который поддерживает служба назначения, и соответствующим образом пересылает запрос. Все коммуникации проходят через служебную шину, что делает ее критически важным посредником; если ESB выходит из строя, это приводит к сбою всей системы, что подчеркивает его сложность и важность.

Этот упрощенный обзор архитектуры ESB также подчеркивает, что, хотя ESB является центральным компонентом, система может включать в себя дополнительные элементы, такие как брокеры доменов, службы данных, службы оркестрации процессов и механизмы правил. Такая конфигурация обеспечивает более высокую производительность и исключает единую точку отказа; если один ESB выходит из строя, это затрагивает только его конкретный бизнес-домен. Этот архитектурный подход имеет множество преимуществ, особенно в сценариях, где мы не владеем веб-службами и нам требуется посредник для передачи сообщений между службами, организации бизнес-процессов с участием нескольких веб-служб и выполнения других промежуточных задач. Стоит также отметить, что современные реализации ESB значительно изменились и часто включают в себя пользовательские интерфейсы с поддержкой перетаскивания для настройки, что упрощает их развертывание и управление.

При построении облачных приложений обобщенный посредник может быть реализован в виде **облачного сервиса (Cloud Services),** который обеспечивает взаимодействие между различными компонентами приложения, работающими на разных узлах облака.

В **микросервисной архитектуре (Microservices)** каждый компонент системы представлен как отдельный микросервис, который взаимодействует с другими микросервисами через обобщенный посредник – например, централизованный сервис реестра. Благодаря микросервисам был осуществлен переход в парадигме SOA от удаленного вызова методов объекта (CORBA) к передаче сообщений между сервисами. Однако

нужно понимать, что в рамках SOA веб-сервисы – не просто API общего назначения, всего лишь предоставляющие CRUD-доступ к базе данных через HTTP. В каких-то случаях эта реализация может быть полезной, но ради целостности данных необходимо, чтобы пользователи понимали лежащую в основе реализации модель и соблюдали бизнес-правила. SOA подразумевает, что веб-сервисы являются ограниченными контекстами бизнес-субдоменов (business sub-domain) и отделяют реализацию от решаемых веб-сервисами задач.

Основной принцип микросервисной архитектуры – это независимость каждого микросервиса, что обеспечивает большую гибкость и масштабируемость системы. Каждый микросервис можно разрабатывать, внедрять и масштабировать независимо от других, что позволяет использовать различные технологии и быстро адаптироваться к изменяющимся бизнес-требованиям. Тем не менее такая гибкость влечет за собой и новые сложности. Например, управление распределенными транзакциями и согласованностью данных становится более сложным, так как одной операции могут требоваться несколько микросервисов. Поэтому для обеспечения надежности и производительности системы необходимо внедрять продвинутые механизмы мониторинга, централизованного логирования и управления ошибками.

Очередь сообщений. При таком подходе приложения могут осуществлять передачу данных и вызовов с помощью платформонезависимых сообщений. Приложениям не нужно знать о других приложениях, а необходимо только уметь общаться на одном языке с очередью. При таком подходе программу назначения определяет менеджер очереди, что обеспечивает легкость в модернизации и замену отдельных элементов программного кода. Очередь сообщений использует в качестве компонента инфраструктуры программный брокер сообщений (RabbitMQ, Beanstalkd, Kafka и т. д.). Для реализации связи между приложениями можно по-разному настроить очередь:

- Запрос/ответ. Обычно такой поход применяется при клиент-серверном общении. Клиент создает запрос-сообщение с ссылкой на «конфу» (conversation reference). Очередь по данному маркеру определяет пункт назначения, который в обратном направлении, зная conversation reference, присылает ответ. При этом возможно создание длинных цепочек запросов-ответов, соответствующих продолжительным бизнес-процессам (цепочек событий, sagas).
- Публикация/подписка. По спискам. В основе лежит подписка программ на темы (topics) в брокере очередей сообщений. В таком случае сообщение отправляется в определенную тему по определенному набору значений критериев, т. е. тип сообщения может быть как задан вручную, так и определен автоматически. Далее происходит рассылка сообщения всем по списку подписчиков на тему.

- На основе вещания. Сообщение отправляется непосредственно в очередь, а очередь осуществляет широковещательную рассылку этого сообщения. Узлы-программы сами фильтруют данные и обрабатывают только подходящие сообщения.

Все эти паттерны можно отнести к либо к pull- (polling), либо к push-подходу. В pull-сценарии очередь ничего не знает или не может обратиться к клиентам и все программы-клиенты сами должны опрашивать очередь на наличие новых сообщений. Задержка в передачи сообщения зависит от частоты опроса очереди. В push-сценарии очередь знает обо всех клиентах и сразу же отдает клиентам сообщения в момент его поступления. Задержки нет, но клиенты не управляют своей нагрузкой, т. е. клиент может выполнять другие функции, но обязан отреагировать на сообщение от очереди.

Выбор конкретного способа реализации обобщенного посредника зависит от требований конкретной системы, ее архитектуры, масштаба и других факторов. Важно выбрать наиболее подходящий подход для конкретной задачи, чтобы обеспечить эффективное взаимодействие компонентов системы и удовлетворить требования к производительности, масштабируемости и надежности.

Иногда модулю необходимо только информировать другие модули о произошедших событиях или изменениях внутри себя. Ему не важно, будет ли использоваться эта информация. В таких сценариях модули могут обмениваться сообщениями или событиями, не имея прямых ссылок друг на друга.

Связь между модулями посредством передачи сообщений слабее, чем прямые зависимости, и обычно реализуется с использованием шаблонов проектирования «наблюдатель» и «медиатор».

«Наблюдатель»: используется при наличии зависимости «один ко многим», когда несколько модулей зависят от состояния одного основного модуля. Это включает механизм уведомления, при котором основной модуль отправляет одинаковые сообщения всем своим подписчикам.

Модули, заинтересованные в этих обновлениях, реализуют интерфейс «подписчик» и подписываются на список уведомлений. Этот шаблон часто используется в структурах пользовательского интерфейса, позволяя ядру приложения (модели) оставаться независимым от реализации других модулей, в то же время информируя подключенные интерфейсы об изменениях состояния, требующих обновления.

Кроме того, организация взаимодействия посредством передачи сообщений имеет то преимущество, что не требует присутствия «подписчиков» для «опубликованных» сообщений.

Если «подписчик» отсутствует, то сообщение просто не будет доставлено.

«Медиатор» (рис. 4.1) применяется, когда между модулями существует зависимость «многие ко многим».

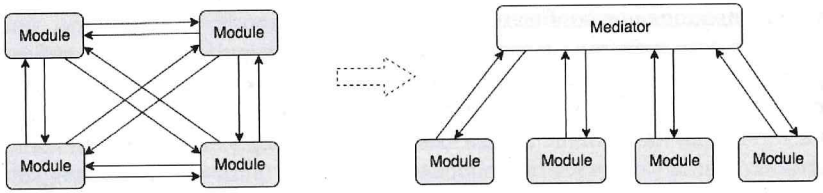


Рис. 4.1. Использование паттерна «медиатор» для ослабления зависимостей

Посредник действует как центральный коммуникационный узел и устраняет необходимость в том, чтобы модули напрямую ссылались друг на друга. Посредник эффективно инкапсулирует взаимодействие между несколькими модулями.

Расширяя эти шаблоны, механизмы передачи сообщений могут значительно повысить модульность и разделение задач внутри системы. Разделяя модули и облегчая асинхронную связь, они способствуют независимому развитию и обслуживанию компонентов системы. Более того, эти шаблоны поддерживают масштабируемость, позволяя распределять модули по нескольким машинам или процессам, повышая общую надежность и производительность системы.

Такой подход к проектированию имеет решающее значение в современных архитектурах программного обеспечения, включая микросервисы, системы, управляемые событиями, и приложения реального времени, где модули должны взаимодействовать эффективно и надежно, не будучи тесно связанными друг с другом.

4.6. Закон Деметры (law of Demeter)

Закон Деметры, также известный как принцип минимального знания, является важным принципом объектно-ориентированного программирования. Он утверждает, что объект A не должен иметь возможность получить непосредственный доступ к объекту C, если у объекта A есть доступ к объекту B и у объекта B есть доступ к объекту C. Это означает, что все зависимости в коде должны быть явными и классы или модули могут использовать только свои зависимости.

При применении закона Деметры связанность уменьшается за счет обращения к соседним классам. Принцип минимального знания, на котором основан закон Деметры, гласит, что объект или модуль должен знать как можно меньше деталей о структуре и свойствах других объектов или модулей.

Для наглядности рассмотрим пример на языке Python:

```
class a:
    def __init__(self, b):
        self.b = b

    def do_something(self):
        self.b.do_something_with_c()

class b:
    def __init__(self, c):
        self.c = c

    def do_something_with_c(self):
        self.c.do_something()

class c:
    def do_something(self):
        print("Doing something in class C")

# Создание объектов
c_instance = C()
b_instance = B(c_instance)
a_instance = A(b_instance)

# Вызов метода do_something() у объекта класса A
a_instance.do_something()
```

4.7. SOLID

Роберт Мартин, также известный как Uncle Bob, составил список из пяти принципов SOLID, которые стали основой для хорошего объектно-ориентированного проектирования. Эти принципы включают:

4.7.1. Single Responsibility Principle (принцип единственной ответственности)

Не должно существовать более одного мотива для изменения данного класса. Об этом принципе говорят, что он является одним из простейших для понимания, но достаточно сложным для того, чтобы легко научиться им правильно пользоваться. Рассмотрим пример. Пусть у нас есть класс, реализующий некоторую функциональность, связанную с банковским счетом:

```
class Account : ActiveRecord
```

```
{
    public Guid Id{ get{ ... } }
    public string Number { get{ ... } }
    public decimal CurrentBallance { get { ... } }
    public void Deposit(decimal amount){ ... }
    public void Withdraw(decimal amount){ ... }
    public void Transfer(decimal amount, Account recipient){ ... }
    public TaxTable CalculateTaxes(int year){ ... }
}
```

Как видим, класс несет ответственность:

- за персистентность;
- логику управления балансом;
- логику расчета налогов.

Все эти задачи и являются теми самыми мотивами, которые влияют на жизненный цикл класса. Проведя рефакторинг, можно получить следующий код:

```
class Account
{
    public string Number { get{ ... } }
    public decimal CurrentBallance { get { ... } }
    public void Deposit(decimal amount){ ... }
    public void Withdraw(decimal amount){ ... }
    public void Transfer(decimal amount, Account recipient){ ... }
}

class AccountRepository
{
    public Account GetByNumber(string number){ ... }
    public void Save(Account acc){ ... }
}

class TaxCalculator
{
    public TaxTable CalculateTaxes(Account acc, int year){ ... }
}
```

Сложность в применении данного принципа заключается в том, что прежде всего нужно научиться правильно понимать границы его использования. Ведь даже в приведенном примере паттерн Active Record превращен в антипаттерн, разом перечеркнув все примеры его успешного применения.

4.7.2. Open/Closed Principle (принцип открытия/закрытия)

Объекты проектирования (классы, функции, модули и т. д.) должны быть открыты для расширения, но закрыты для модификации. Иными словами, нужно избегать случаев, когда появление новых требований к функциональности влечет за собой модификацию существующей логики, стараясь реализовать возможность ее расширения.

Рассмотрим простой пример. Пусть у нас в системе есть некий класс, отвечающий за просмотр логов:

```
class LogViewer
{
    public IEnumerable<Transaction> GetByDate(DateTime dateTime){
... }
}
```

Пусть возникла необходимость реализовать возможность выборки транзакций по имени пользователя, класс может быть модифицирован следующим образом:

```
class LogViewer
{
    public IEnumerable<Transaction> GetByDate(DateTime dateTime){
... }
    public IEnumerable<Transaction> GetByUser(string name){ ... }
    public IEnumerable<Transaction> GetByDateAndUser(DateTime
dateTime, string name){ ... }
}
```

Подобная эволюция дизайна является типичным примером нарушения принципа открытия/закрытия. А типичным решением этой проблемы мог бы стать следующий код:

```
class LogViewer
{
    public IEnumerable<Transaction> GetTransaction(GetSpecification
spec){ ... }
}

abstract class GetSpecification
{
    public GetSpecification CombineWith(GetSpecification nextSpec){ ... }
    // ...
}

class GetByDateSpecification : GetSpecification
{
    // ...
```

```
}

class GetByUserSpecification : GetSpecification
{
  // ...
}

// Пример использования
class Client
{
  public void ShowLog()
  {
    var viewer = new LogViewer();
    var transactions = viewer.GetTransaction(
      new GetByDateSpecification()
      .CombineWith(new GetByUserSpecification()));
  }
}
```

Таким образом, можно констатировать, что объект, спроектированный по принципу открытости/закрытости, обладает следующими характеристиками:

- «Открыто для расширения»: его поведение можно улучшить (расширить) путем добавления новых объектов, которые привносят новые поведенческие аспекты;
- «Закрыто для модификации»: при расширении его поведения нет необходимости изменять исходный код.

Используя интерфейсы и полиморфизм, мы можем придерживаться принципа открытости/закрытости при разработке программного обеспечения. Это означает, что мы можем обновлять и дополнять код, не изменяя существующую функциональность. Пока взаимодействие между модулями определяется через интерфейсы и не зависит от конкретных реализаций, один модуль можно заменить другим, соответствующим тому же интерфейсу, или можно добавить новые модули для улучшения функциональности без негативного влияния на систему в целом.

4.7.3. Liskov Substitution Principle (принцип замещения Лисков)

Функции, которые используют ссылки на базовые классы, должны иметь возможность использовать объекты производных классов, не зная об этом. Впервые этот принцип был упомянут Барбарой Лисков в 1987 г. на научной конференции, посвященной объектно-ориентированному программированию. Он является важнейшим критерием для оценки качества принимаемых решений при построении иерархий

наследования. Сформулировать его можно в виде простого правила: тип S будет подтипом T тогда и только тогда, когда каждому объекту oS типа S соответствует некий объект oT типа T таким образом, что для всех программ P, реализованных в терминах T, поведение P не будет меняться, если oT заменить на oS.

Классическим примером нарушения этого принципа является построение иерархии такого рода:

```
class Rectangle
{
  public virtual int Width { get; set; }
  public virtual int Height { get; set; }

  public int CalculateRectangleArea()
  {
    return Width*Height;
  }
}

class Square : Rectangle
{
  public override int Height
  {
    get{ return base.Height; }
    set
    {
      base.Height = value;
      base.Width = value;
    }
  }

  public override int Width
  {
    get{ return base.Width; }
    set
    {
      base.Width = value;
      base.Height = value;
    }
  }
}

class Program
{
```

```
    private static Rectangle CreateRecatgle()
    {
        return new Square();
    }

    static void Main()
    {
        Rectangle r = CreateRecatgle();
        r.Width = 3;
        r.Height = 2;

        Assert.AreEqual(6, r.CalculateRectangleArea());
    }
}
```

Этот пример заставляет задуматься о том, что такое декларация типа в терминах объектно-ориентированного языка программирования, который мы используем: достаточно ли нам описать интерфейс объекта с помощью обычного абстрактного класса со списком методов, типами параметров и возвращаемого значения? каким образом мы можем декларировать требования к значениям параметров метода и свойства, которыми будет обладать возвращаемое значение? как нам описать исключения, которые может сгенерировать метод во время выполнения? как нам описать изменение состояния объекта на разных этапах его жизненного цикла? Задавая себе эти вопросы и находя ответы, можно спроектировать систему, которая действительно будет удовлетворять принципу замещения Лисков.

4.7.4. Interface Segregation Principle (принцип изоляции интерфейса)

Клиент не должен вынужденно зависеть от элементов интерфейса, которые он не использует (рис. 4.2).

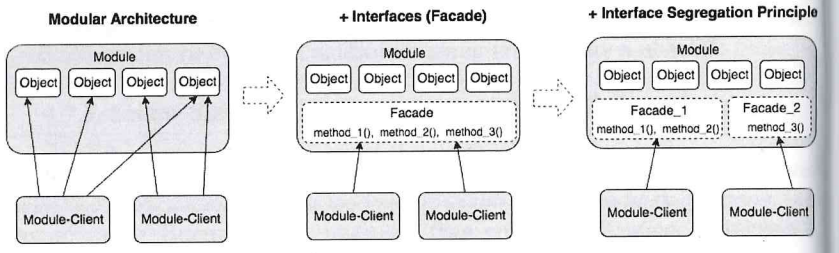


Рис. 4.2. Пример соответствия принципу изоляции интерфейса с применением паттерна «фасад»

Другими словами, этот принцип можно сформулировать так: зависимость между классами должна быть ограничена как можно более узким интерфейсом. Пример нарушения этого принципа:

```
abstract class ServiceClient
{
    public string ServiceUri{ get; set; }
    public abstract void SendData(object data);
    public abstract void Flush();
}

class HttpServiceClient : ServiceClient
{
    public override void SendData(object data)
    {
        var channel = OpenChannel(ServiceUri);
        channel.Send(data);
    }

    public override void Flush()
    {
        // Метод ничего не делает, но присутствует в классе
    }
}

class BufferingHttpServiceClient : ServiceClient
{
    public override void SendData(object data)
    {
        Buffer.Write(data);
    }

    public override void Flush()
    {
        var channel = OpenChannel(ServiceUri);
        channel.Send(Buffer.GetAll());
    }
}
```

Решение этой проблемы заключается в проектировании грамотной иерархии интерфейсов для уменьшения такой зависимости:

```
abstract class ServiceClient
{
    public string ServiceUri{ get; set; }
    public abstract void SendData(object data);
```

```
}

abstract class BufferingServiceClient : ServiceClient
{
   public abstract void Flush();
}

class HttpServiceClient : ServiceClient
{
   public override void SendData(object data){ ... }
}

class BufferingHttpServiceClient : BufferingServiceClient
{
   public override void SendData(object data){ ... }
   public override void Flush(){ ... }
}
```

Громоздкие интерфейсы нарушают данный принцип. Рассмотрим реализацию MembershipProvider для ASP.NET, при этом унаследуемся от стандартного базового класса. MembershipProvider – это абстрактный класс в ASP.NET, который предоставляет базовую функциональность для работы с пользователями и управлениями их учетными записями. При создании собственного класса CustomMembershipProvider, который наследуется от MembershipProvider, разработчики должны реализовать множество методов и свойств, предоставленных базовым классом. Это может привести к созданию громоздкого интерфейса с большим количеством методов, даже если в реальности не все из них нужны для конкретной реализации.

4.6.5. Dependency Inversion Principle (принцип обращения зависимости)

Модули верхних уровней не должны зависеть от модулей нижних уровней. Оба типа модулей должны зависеть от абстракций. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций. Иерархия модулей определяет направление взаимодействия. В форме UML-диаграммы (рис. 4.3) имеется интерфейс модуля (посередине) и модуль, обращающийся к интерфейсу (клиент модуля) – справа и его реализующий – слева.

Рис. 4.3. UML-диаграмма для принципа обращения зависимостей

Принцип инвертирования зависимостей определяет отношение этих трех сущностей так:

- Клиент модуля (справа) ничего не должен знать о реализации модуля (слева). Клиент модуля имеет дело только с интерфейсом (посередине).
- Реализация модуля (слева) ничего не должна знать о клиенте модуля (справа). Реализация модуля реализует определенный интерфейс (посередине).
- Интерфейс (посередине) не должен зависеть от реализации (слева). Сама реализация должна зависеть от интерфейса.

Принцип инвертирования зависимостей является основой для повторного использования кода. Переиспользование кода означает, что абстрактный механизм остается неизменным, но его конкретные составляющие могут меняться. Например, при работе с модулем, который читает, обрабатывает и публикует данные, вы можете использовать интерфейс для соединения с базой данных, не заботясь о том, как именно это соединение установлено. Точно так же при публикации данных вам не важно, куда они отправляются.

Ваш код остается неизменным независимо от того, как именно получено соединение с базой данных. Используя интерфейс, вы отделяете реализацию от абстракции. Точно так же при публикации данных вас не волнует конкретное место, куда они направляются. Важно лишь соблюдение контракта интерфейса.

Ограничивая мир вашего модуля двумя интерфейсами – соединением с базой и публикацией данных, вы создаете возможность для переиспользования кода при различных реализациях этих интерфейсов. Однако важно избегать промахов при определении абстракций.

Ошибка может возникнуть из-за неверного определения абстракции. Например, если считать соединение с базой данных единственным возможным источником данных. Поэтому важно начинать проектирование с максимально абстрактного уровня и определять примитивные абстракции заранее.

Для того чтобы минимизировать ошибки при определении абстракций, следует начинать проектирование с примитивного уровня. Например, определить модуль как получающий данные из источника и выдающий результат в приемник. Далее определить структуру входных данных и операции источника данных согласно бизнес-логике.

Входные данные могут быть абстрактными, но обычно их структура известна хотя бы в общих чертах. Совсем абстрактные данные модуль обработать не может, структура входных данных обусловлена бизнес-логикой. То есть данные можно определить в виде одного класса определенной структуры, InputDataUnit.

Источник данных абстрактен, и мы не должны предполагать ничего определенного. Если нам нужен, например, список записей по параметрам и отдельно запись по идентификатору, то именно эти потребности и станут основой определения операций источника данных. При этом и приемник абстрактен. Мы взаимодействуем с интерфейсом с одной операцией, а его реализация – особенности поведения потребителя данных. Итоговый вариант можно описать (рис. 4.4).

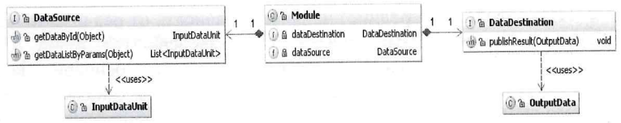


Рис. 4.4. Создание абстрагированных входных и выходных интерфейсов для принципа обращения зависимостей

Абстрагированный входной интерфейс, абстрагированный выходной интерфейс: в этих условиях наш модуль может переиспользоваться без каких-либо проблем. Таким образом, модуль зависит только от абстракций, причем как для выходных, так и для входных данных. Он ничего не знает о том, как устроены реализации этих абстракций. Сами абстракции на реализацию тоже не рассчитаны. Все требования принципа dependency inversion выполнены. Главное – зафиксировать абстракции на этапе проектирования.

Глава 5. CI/CD

Концепция CI/CD (Continuous Integration и Continuous Delivery) – это подход к разработке программного обеспечения, который позволяет значительно ускорить процесс создания продуктов, увеличить скорость сборки, уменьшить число ошибок в proguction и повысить их качество. Основным конкурентным преимуществом в этом подходе являются скорость и надежность сборки.

Continuous Integration (CI) – процесс автоматического тестирования и интеграции каждого изменения кода в общую кодовую базу. Благодаря этому можно оперативно выявлять и устранять конфликты между различными ветвями кода до развертывания продукта, что значительно экономит время на этапе интеграции.

Continuous Delivery (CD) предполагает, что каждое внесенное изменение проходит полное тестирование и сборку. Если все тесты проходят успешно, то приложение автоматически готово для развертывания (deployment) в производственную среду. Этот процесс гарантирует быструю и безопасную доставку обновлений пользователям программного продукта, так как Continuous Deployment (непрерывное развертывание), считаемое частью CD, – это еще более автоматизированный этап CI/CD, при котором успешное финальное тестирование приводит к автоматическому развертыванию приложения. Это позволяет максимально сократить время от внесения изменений до их доставки конечному пользователю, делая процесс внедрения изменений более частым и надежным.

Методология CI/CD подразумевает разделение процесса разработки на семь этапов (рис. 5.1):

Написание кода. Разработчики пишут код для своих модулей и проводят его отладку. После проверки результат объединяется в главной ветви проекта.

Сборка. Система контроля версий автоматически запускает сборку и тестирование проекта. Это может быть настроено с помощью таких инструментов, как Jenkins.

Ручное тестирование. После успешного автоматизированного тестирования код передается для более детального ручного тестирования.

Релиз. Внесенные в процессе тестирования исправления интегрируются, и новая версия кода выпускается для клиентов.

Развертывание. Рабочая версия кода размещается на производственных серверах, и клиенты начинают взаимодействовать с программой.

Поддержка и мониторинг. Начинается стадия эксплуатации продукта конечными пользователями, при этом разработчики продолжают его поддержку и мониторят отзывы пользователей.

Планирование. На основе анализа пользовательского опыта разрабатываются новые функции и составляется план дальнейших доработок, после чего цикл разработки начинается заново. Чаще всего этот процесс запускает новый виток CI/CD, планирование считается первым этапом (см. рис. 5.1).

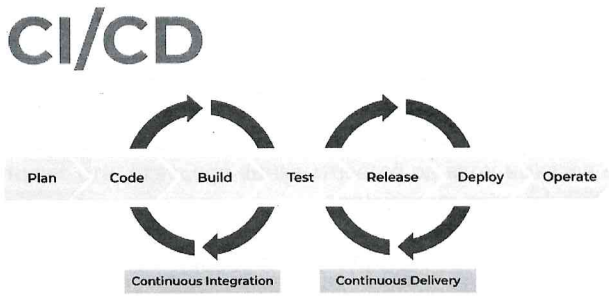


Рис. 5.1. Последовательность этапов CI/CD

CI/CD стала популярной благодаря возможности быстро внедрять обновления программного обеспечения. Как только программисты пишут минимальный функциональный протестированный блок, он попадает к потребителям. Идея CI/CD появилась в 2006 г., к 2008 г. она начала активно применяться, особенно с развитием облачных сервисов. Эта методология подходит для веб-разработки, омниканальных решений, e-commerce и других задач, связанных с frontend- и middleware-компонентами. Однако не всегда внедрение CI/CD оправдано. При работе с CI/CD пользователь постоянно получает микропатчи, а программисты трудятся чаще всего над небольшими доработками. Так, в сферах с редким обновлением программного обеспечения эта методология может не принести значительных выгод. При разработке программного обеспечения могут использоваться различные инструменты для автоматизации процессов тестирования и доставки кода конечным пользователям.

Рассмотрим некоторые из них:

- **GitLab CI.** Эта среда предоставляет возможности управления репозиториями проекта, документирования результатов тестов и функциональности, а также отслеживания ошибок.
- **Docker**. Система для автоматизированного развертывания проектов, поддерживающая контейнеризацию. Она позволяет упаковать проект вместе с его окружением и зависимостями.
- **Travis-CI**. Инструмент, который может подключаться к репозиториям на GitHub с минимальными настройками. Travis-CI является облачным сервисом, поэтому его установка не требуется.

- **Circle-CI**. Продукт использует бесшовную интеграцию с GitHub. Предлагается веб-интерфейс для отслеживания версий сборок и ведения задач. Также решение поддерживает отправку оповещений по почте и другим каналам связи.
- **Jenkins**. Востребованный DevOps-инструмент, позволяющий работать с разными плагинами для гибкой настройки процессов под конкретные задачи разрабатываемого продукта.
- **PHP Censor**. Сервер непрерывной интеграции, предназначенный для автоматизации сборки PHP-проектов. Работает с репозиториями GitLab, GitHub, Mercurial и пр., а также с библиотеками тестирования Atoum, PHP Spec, Behat. Инструмент документирован, но требует самостоятельной настройки и хостинга.
- **SberCloud K8S** (SberCloud Managed Kubernetes). Сервис автоматического развертывания, масштабирования и управления софтом на основе Kubernetes. Позволяет создавать кластеры последних версий, управлять автообновлением, автоматически распределять нагрузку, отслеживать статистику потребления и т. д.

Одной из интересных практических частей для создания и развития процессов CI/CD в команде является настройка автоматической сборки и деплоя приложения на сервер. Для этого можно использовать следующие инструменты:

- Gitlab CI/CD;
- Docker.

Ниже приведен пример .gitlab-ci.yml-файла для настройки автоматической сборки и деплоя приложения на сервер:

```
image: docker:19.03.5

services:
  - docker:19.03.5-dind

before_script:
  - docker login -u "$CI_REGISTRY_USER" -p "$CI_REGISTRY_PASSWORD" $CI_REGISTRY

build_job:
 script:
  - docker build -t $CI_REGISTRY_IMAGE .
  - docker push $CI_REGISTRY_IMAGE

deploy_job:
 script:
  - ssh user@server "docker pull $CI_REGISTRY_IMAGE"
```

```
    - ssh user@server "docker rm -f app || true"
    - ssh user@server "docker run -d --name app -p 80:80
$CI_REGISTRY_IMAGE"

stages:
 - build
 - deploy

build:
 extends: .build_job
 stage: build

deploy:
 extends: .deploy_job
 stage: deploy
```

В примере используется Gitlab CI/CD, который имеет интеграцию с Docker для сборки контейнера в Docker и его отправки в Docker registry. Далее на сервере развернут Docker, в который с помощью SSH подключения выполняются команды на деплой созданного образа и его запуск на сервере.

Для запуска этого процесса нужно настроить сервер для выполнения команд по ssh, создать в Gitlab CI/CD переменные $CIREGISTRYUSER, $CIREGISTRYPASSWORD, $CIREGISTRYIMAGE, в которые будем помещать логин, пароль и путь до созданного в Docker registry образа приложения.

Вторым примером послужит настройка автоматического тестирования приложения. Это позволяет не только быстро и эффективно проверять каждое новое изменение в коде программы, но также минимизировать количество ошибок и сбоев в работе готового продукта.

Для демонстрации данного процесса рассмотрим пример создания автоматических тестов для простого веб-приложения на языке программирования Python с использованием фреймворка Flask. Для начала необходимо создать файлы и настроить окружение для тестирования. Создадим три файла: app.py (главный файл приложения), apptest.py (файл с тестами) и requirements.txt (файл для установки необходимых библиотек). В файле app.py напишем простое веб-приложение на Flask, которое будет отображать текст «Hello, world!» при запросе пользователем.

```
from flask import Flask

app = Flask(name)

@app.route("/")
def home():
```



```
    return "Hello, world!"

if name == "main":
    app.run(debug=True)
```

В файле apptest.py напишем тесты для приложения. В данной реализации проверяем, что при запросе на корневой URL сервер возвращает строку «Hello, world!»

```
import unittest
from app import app

class TestApp(unittest.TestCase):

    def test_home(self):
        with app.test_client() as client:
            response = client.get('/')
            self.assertEqual(response.status_code, 200)
            self.assertEqual(response.get_data().decode(), 'Hello, world!')

if __name__ == '__main__':
    unittest.main()
```

Чтобы код работал по определенным правилам, добавим файл .lint в корневую папку проекта. В файле .lint пропишем правила линтера, с которыми мы будем сравнивать код. В данном примере мы используем линтер flake8, который проверяет PEP8-стандарт кода:

```
[flake8]
exclude = .git,__pycache__,venv
max-line-length = 120
```

Теперь можем настроить автоматическое тестирование с последующей сборкой и деплоем приложения. Для этого используем сервис CircleCI. Создайте файл .circleci/config.yml в корне проекта со следующим содержимым:

```
version: 2
jobs:
 build:
  docker:
   - image: circleci/python:3.8.0
  steps:
   - checkout
   - run: pip install -r requirements.txt
   - run: pytest app_test.py
     name: Test
   - run:
```

```
name: Flake8
command: |
  pip install flake8
  flake8 .
```

В данном примере мы используем Docker-контейнер circleci/python:3.8.0 для сборки и тестирования нашего приложения. После установки необходимых библиотек и запуска тестов мы также проверяем код на соответствие PEP8-стандартам с помощью линтера flake8. Когда теперь мы запустим изменения в наш репозиторий на GitHub, CircleCI автоматически запустит все тесты и линтеры и опубликует нашу программу на веб-странице Heroku, если они успешно пройдены.

Глава 6. РЕФАКТОРИНГ КОДА

Красивый код – это хорошо написанный и структурированный код, безразличный компилятору. Для компилятора или в процессе исполнения важны структурированность и эффективность программы для машины, компьютера. Именно люди пишут код, вносят в него изменения, модифицируют, к тому же удобство работы с кодом программистов напрямую влияет на скорость и качество внесения изменений в программный продукт. При этом важно понимать статичность написанного кода, так как он протестирован и обеспечивает полную функциональность и работоспособность на предыдущем этапе и динамичность нового кода. Если в результате добавления нового кода, новой функциональности необходимо изменение старого кода, то он становится динамичным, требует тестирования. Более того, изменение старого кода влечет за собой возможные проблемы в связанных с ним блоках до возможной полной неработоспособности программного обеспечения.

Если при добавлении новой функциональности, нового кода структура существующего кода неудобна и требует изменений, то следует перед написанием нового кода провести рефакторинг старого кода. Рефакторинг направлен на улучшение структуры кода, его расширяемости и в общем виде упрощении дальнейшего взаимодействия программистов с данным кодом. В отличие от рефакторинга **оптимизация производительности** фокусируется на улучшении скорости работы компонента, не меняя его задач. Хотя обе техники могут не изменять видимого для пользователя поведения программы, их цели различны: рефакторинг направлен на упрощение понимания кода, в то время как оптимизация производительности **улучшает скорость работы, что может затруднить понимание кода**, но необходимо для достижения желаемого результата. Например, максимальную эффективность со стороны вычислительной машины будет иметь код, написанный без каких-либо абстракций на языке программирования assembler.

Внешний вид и функциональность программы по результатам рефакторинга не изменяются, поэтому удобно разнести процессы изменения структуры имеющегося программного кода и добавления новой функциональности. При этом добавление новых функций не должно приводить к изменениям в структуре существующего кода, новые возможности просто добавляются. Тогда в полной мере выполняется принцип открытия/закрытия (Open/Closed Principle). Однако в процессе разработки может возникнуть необходимость переключаться между двумя видами работы. При попытке добавить новую функцию иногда становится ясно, что изменение структуры кода сделает эту задачу гораздо проще. В таких случаях целесообразно временно переключиться на ре-

факторинг. После улучшения структуры кода можно продолжить работу над добавлением новой функциональности.

При добавлении новой функциональности и необходимости изменить старый код часто рефакторинг не проводится. Это связано с тем, что рефакторинг – трудоемкий и долгий процесс, не приносящий прибыли фирме и не увеличивающий функциональности для пользователя. При таком подходе возможно изменение таких модулей, которые влияют на уже работающий функционал, и некоторые функции программного обеспечения (реализованные ранее) могут отказать. И если ошибку, которая влияет на часто используемую функциональность, найдут и быстро исправят, то ошибки в модулях, запускающихся один раз в неделю/месяц и т. п., могут серьезно навредить программному обеспечению, так как они не будут своевременно обнаружены. Более того, потеря структурированности кода со временем накапливается. Если программисту сложно разобраться в запутанном коде с множеством связей, то ему трудно его поддерживать и появляется желание полностью переписать модуль.

Так как рефакторинг изменяет уже работающий код, то необходимо проводить тестирование (автоматическое) для поддержания уверенности, что никакая другая функциональность не откажет. Такой подход требует, чтобы новая функциональность была внесена после того, как будут реализованы необходимые тесты. Подобные действия требуют организационных мер. В гибких методологиях (например, экстремальном программировании) рефакторинг внедряется в технологические процессы. При этом автоматическое юнит-тестирование позволяет убедиться, что рефакторинг не разрушил существующую функциональность.

Технический долг – метафора, придуманная Уордом Каннингемом для OOPSLA в 1992 г. В современном мире множество исследователей развили и расширили данную метафору, предложив свои концепции. Понятие долга в термине «технический долг» – это аналог из сферы финансов. Когда добавляется новая функциональность, у старого написанного кода берется долг/кредит и впоследствии придется оплатить проценты по долгу при каждом использовании функций. Чем больше используется какая-нибудь функция, тем чаще она выступает в качестве кредитора. Если такая функция написана плохо и ее изменение дает большие проценты, то и выплаты при использовании данной функции будут большими. Что может программист брать в долг? В долг у кодовой базы программист берет время на разработку.

Допустим, для добавления новой функциональности требуется использовать сложный по своей структуре и правилам взаимодействия модуль существующего кода. Если бы структура данного модуля была бы простой, удобной и понятной для программиста, то работа по добавлению новых функций заняла бы 6 дней. Со сложным модулем работа предположительно займет 8 дней, так как придется разбираться с запу-

танным существующим функционалом, т. е. 2 дня задержки вызваны процентами по техническому долгу.

Возможен вариант уменьшения долга, например, можно исправить, переписать модуль, но это потребует работы на 4 дня и уменьшит долг до 1 дня, итого 6 + 4 + 1 = 11 дней. По сравнению с 8 днями работы с долгом переписывать код невыгодно и проще смириться со сложной структурой старого кода. Однако если таких модулей необходимо написать два, то суммарно без переписывания долга потребуется 8 · 2 = 16 дней, а с переписыванием: 4 + 7 · 2 = 18 дней. Аналогично для трех модулей. Для четырех модулей будет эффективно переписать код, уменьшить долг (8 · 4 = 32 против 4 + 7 · 4 = 32), т. е. эффективно оплачивать такой долг для часто используемого модуля. И это для примера, в котором оплата долга не сильно уменьшает проценты по кредиту. Если же проценты составят 6 дней, то даже для одного применения модуля эффективно проводить рефакторинг.

Данная метрика при своей простоте основана на оценочных метриках, и точность таких оценок очень низкая. Тем не менее данная метрика понятна менеджеру и может быть использована для доказательства необходимости рефакторинга.

Существует и другой подход, навеянный экономикой, – постепенно выплачивать долг. То есть при добавлении новой функциональности немного времени тратится на уплату долга и тогда проценты по долгу не растут в катастрофическом объеме. Но следует учитывать, что большее время все еще требуется тратить на часто используемые области кода, так как долг важен не только размером, но и частотой обращения. Таким образом, неструктурированные, но стабильные и редко изменяемые области кода можно оставить в покое. В отличие от стабильных частей кода области высокой активности требуют нулевой терпимости к мусору, потому что там процентные платежи чрезвычайно высоки.

Отдельно стоит отметить, что полученный в результате рефакторинга исходный код – не цель или ценность. Эстетически или художественно код может представлять некоторый интерес, как, например, быстрый обратный квадратный корень (также быстрый InvSqrt() или 0x5F3759DF), но это исключения. В общем случае код – инструмент для создания программного продукта, которым кто-либо пользуется, и эстетика в первую очередь заключается в скорости и качестве программного обеспечения. Наиболее полную классификацию описал Мартин Фаулер в своей книге[1], в рамках которой приведены 8 крупных классов методов рефакторинга, разбитых на более мелкие элементы.

[1] См.: Фаулер М. Рефакторинг: улучшение существующего кода. – СПб. : Символ-Плюс, 2003.

В книге М. Фаулера для каждого элемента приведены примеры код и структур данных, но из-за ограниченности объемов приведем только перечисление возможных методов рефакторинга с указанием английски названий для удобства поиска:

- составление методов;
- выделение метода (Extract Method);
- встраивание метода (Inline Method);
- встраивание временной переменной (Inline Temp);
- замена временной переменной вызовом метода (Replace Tem with Query);
- введение поясняющей переменной (Introduce Explainin Variable);
- расщепление временной переменной (Split Temporary Variable);
- удаление присваиваний параметрам (Remove Assignment to Parameters);
- замена метода объектом методов (Replace Method with Metho Object);
- замещение алгоритма (Substitute Algorithm);
- перемещение функций между объектами;
- перемещение метода (Move Method);
- перемещение поля (Move Field);
- выделение класса (Extract Class);
- встраивание класса (Inline Class);
- сокрытие делегирования (Hide Delegate);
- удаление посредника (Remove Middle Man);
- введение внешнего метода (Introduce Foreign Method);
- введение локального расширения (Introduce Local Extension);
- организация данных;
- самоинкапсуляция поля (Self Encapsulate Field);
- замена значения данных объектом (Replace Data Value wit Object);
- замена значения ссылкой (Change Value to Reference);
- замена ссылки значением (Change Reference to Value);
- замена массива объектом (Replace Array with Object);
- дублирование видимых данных (Duplicate Observed Data);
- замена однонаправленной связи двунаправленной (Chang Unidirectional Association to Bidirectional);
- замена двунаправленной связи однонаправленной (Chang Bidirectional Association to Unidirectional);
- замена магического числа символической константой (Replac Magic Number with Symbolic Constant);

- инкапсуляция поля (Encapsulate Field);
- инкапсуляция коллекции (Encapsulate Collection);
- замена записи классом данных (Replace Record with Data Class);
- замена кода типа классом (Replace Type Code with Class);
- замена кода типа подклассами (Replace Type Code with Subclasses);
- замена кода типа состоянием/стратегией (Replace Type Code with State/Strategy);
- замена подкласса полями (Replace Subclass with Fields);
- упрощение условных выражений;
- декомпозиция условного оператора (Decompose Conditional);
- консолидация условного выражения (Consolidate Conditional Expression);
- консолидация дублирующихся условных фрагментов (Consolidate Duplicate Conditional Fragments);
- удаление управляющего флага (Remove Control Flag);
- замена вложенных условных операторов граничным оператором (Replace Nested Conditional with Guard Clauses);
- замена условного оператора полиморфизмом (Replace Conditional with Polymorphism);
- введение объекта Null (Introduce Null Object);
- введение утверждения (Introduce Assertion);
- упрощение вызовов методов;
- переименование метода (Rename Method);
- добавление параметра (Add Parameter);
- удаление параметра (Remove Parameter);
- разделение запроса и модификатора (Separate Query from Modifier);
- параметризация метода (Parameterize Method);
- замена параметра явными методами (Replace Parameter with Explicit Methods);
- сохранение всего объекта (Preserve Whole Object);
- замена параметра вызовом метода (Replace Parameter with Method);
- введение граничного объекта (Introduce Parameter Object);
- удаление метода установки значения (Remove Setting Method);
- сокрытие метода (Hide Method);
- замена конструктора фабричным методом (Replace Constructor with Factory Method);
- инкапсуляция нисходящего преобразования типа (Encapsulate Downcast);

– замена кода ошибки исключительной ситуацией (Replace Error Code with Exception);
– замена исключительной ситуации проверкой (Replace Exception with Test);
– решение задач обобщения;
– подъем поля (Pull Up Field);
– подъем метода (Pull Up Method);
– подъем тела конструктора (Pull Up Constructor Body);
– спуск метода (Push Down Method);
– спуск поля (Push Down Field);
– выделение подкласса (Extract Subclass);
– выделение родительского класса (Extract Superclass);
– выделение интерфейса (Extract Interface);
– свертывание иерархии (Collapse Hierarchy);
– формирование шаблона метода (Form Template Method);
– замена наследования делегированием (Replace Inheritance with Delegation);
– замена делегирования наследованием (Replace Delegation with Inheritance)
– крупные рефакторинги;
– разделение наследования (Tease Apart Inheritance);
– преобразование процедурного проекта в объекты (Convert Procedural Design to Objects);
– отделение предметной области от представления (Separate Domain from Presentation);
– выделение иерархии (Extract Hierarchy).

Идеальным можно назвать случай, когда: продукт начинается с нуля, нет предыдущих версий и не требуется совместимость; задача, которую нужно решить, полностью известна и понятна; финансирование проекта гарантировано до достижения результатов.

При разработке такого программного обеспечения можно не заботиться о рефакторинге при создании хорошей, правильной архитектуры. Но обычно программистам требуется дорабатывать существующий проект со своей архитектурой в ограниченные сроки и с постоянно меняющимися задачами. При таком подходе проще написать новое программное обеспечение, чем бороться с техническим долгом, оставшимся от написанного кода, но не понятно, смогут ли программисты реализовать всю существующую функциональность при ограниченном финансировании и времени и еще добавить новую.

Другим вариантом является использование некоторых частей модулей существующего программного обеспечения. Повторное использование кода здесь может показаться удобным, но со временем это может

привести к разрастанию ошибок, разрушению архитектуры программы и дополнительным затратам на модификацию. Кроме того, необходимо разбираться в чужом коде.

Рефакторинг является «золотой серединой» между этими двумя подходами. Он позволяет изменить существующее программное обеспечение, сделав его структуру более понятной, развивая компоненты для повторного использования, улучшая архитектуру программы и облегчая добавление новых функций. Тем не менее рефакторинг не применяется к студенческим работам и редко применяется к большим реальным проектам, и этому есть причины:

- Незнание и непонимание подхода: при рефакторинге не понятно, с чего начать, что должно означать окончание и когда остановиться. Кажется, что можно улучшать код постоянно.
- Отсутствие выгоды здесь и сейчас. Если вы реализовываете новую функциональность, то вы замечаете, что появились новые функции и ими пользуются пользователи. От оптимизации кода растет скорость работы программы, а вот рефакторинг – это вклад на перспективу. Сложно оценить, что добавление новой функции потребовало меньше времени, потому что код был переработан, проведен процесс рефакторинга.
- Приоритет новых функций: если вам оплачивают за добавление новых функций, а не за улучшение существующего кода, то может возникнуть соблазн отложить рефакторинг в сторону. Однако стоит помнить, что неконтролируемый рост кодовой базы без рефакторинга может привести к сложностям и затруднениям в будущем.
- Риск нарушения работы программы: существует опасность, что рефакторинг может привести к ошибкам и нарушениям работы уже существующей программы. Это может вызвать страх перед изменениями и отговорить от рефакторинга.

Несмотря на эти причины рефакторинг является важным инструментом для поддержания здоровой и эффективной кодовой базы, улучшения архитектуры программы и облегчения дальнейшей разработки. Поэтому стоит взвешенно подходить к решению о применении рефакторинга и учитывать его плюсы и минусы.

Глава 7. CODE REVIEW

Code review – практика, которая заключается в тщательном анализ кода, написанного одним разработчиком, другими членами команды. Це лью code review является улучшение качества кода, выявление потенци альных ошибок и уязвимостей, обмен знаниями и опытом между разра ботчиками, а также поддержание стандартов кодирования в проекте.

Процесс code review начинается с того, что разработчик создает за прос на проверку кода (pull request) в системе контроля версий. Други члены команды просматривают изменения, оставляют комментарии предлагают улучшения. После того как все замечания учтены, код може сливаться в основную ветвь проекта. Один из ключевых аспектов cod review – это обсуждение кода между разработчиками, что позволяет н только исправить ошибки, но и обменяться знаниями и опытом. Кром того, code review способствует повышению ответственности разработчи ков за свой код и за работу команды в целом.

Процесс code review также помогает поддерживать стандарты коди рования в проекте. Разработчики могут обсудить различия в стилях ко дирования и прийти к общему соглашению о том, какой стиль будет ис пользоваться в проекте. Процесс code review также способствует повы шению профессионального роста разработчиков, которые могут полу чить обратную связь от более опытных коллег, узнать новые подходы решению задач и улучшить свои навыки программирования. Разработчи ки могут обсудить проблемы и решения, выразить свои мысли и иде которые способствуют более эффективной работе команды. Процес code review способен улучшить коммуникации в команде и повысить до верие к коду. Когда разработчики знают, что их код будет просмотре другими членами команды, они стараются писать более качественный надежный код.

Основными участниками code review являются автор, который пи сал код и отправил его на code review, и рецензент, который анализирус код. Рецензент принимает решение о том, когда код готов для добавле ния в кодовую базу проекта. В самом процессе code review на разны стадиях могут участвовать несколько рецензентов, но в общем виде о может быть один. Началом code review является отправка кода от автор рецензенту, который по итогам проверки может отправить код на дора ботку, указав замечания.

В отличие от часто применяемого при проверке студенческого про граммного обеспечения подхода, когда преподаватель сразу отмечает вс ошибки в программном коде студента, а после исправления студенто дополнительные замечания преподавателя воспринимаются крайне нега тивно, при code review предполагается итерационное (в несколько рау дов) доведение кода до допустимого качества. Каждый раунд – это оди

полный цикл приема-передачи документов и кода между автором и рецензентом: автор присылает изменения в коде или код целиком, рецензент отвечает отзывом на эти изменения. Code review – обычно долгий процесс, который завершается сообщением LGTM («looks good to me»), где рецензент одобряет изменения.

Трудности возникают, когда автор полностью уверен в качестве своего кода и тратить время на исправления, которые автор может считать не важными, автору не хочется. В результате может возникать конфликт между автором и рецензентом, который называется тупиком / тупиковой ситуацией. Для устранения тупиковой ситуации рецензенту необходимо быть деликатным в написании замечаний и общении с автором, поэтому представим хорошие техники code review:

Отдайте компьютерам «скучную» часть работы.

Не все замечания требуют внимания. Если автор неверно идентифицирует переменные, то не следует в каждой переменной отмечать данный недостаток. Необходимо еще до code review выполнить проверку кода, например, используя Hook перед коммитами в Git. Причем данные проверки необходимо автоматизировать, так как если процессы проверки будут запускаться вручную, то неизбежны ошибки, связанные с невнимательностью автора. Следует помнить, что code review – завершающий этап выполнения задачи и для ее закрытия требуется одобрение изменений, поэтому автор пытается как можно быстрее приступить к code review.

Оформите аргументы по стилю в виде руководства по стилю. Хорошее руководство по стилю включает не только правила форматирования, такие как соглашения об именах и отступы, но и указывает на правильное использование функций конкретного языка программирования. Например, в языках программирования JavaScript и Perl существует множество способов реализации одной и той же логики. Руководство по стилю определяет единственно правильный подход к программированию, чтобы избежать разногласий в команде по поводу использования различных функций языка. После внедрения руководства по стилю больше не нужно тратить время на обсуждения и споры по поводу названий файлов или других аспектов форматирования кода. Просто следуя руководству, можно продолжать работу без лишних препирательств. Если какой-то вопрос не описан в руководстве, то лучше не спорить о нем. Вместо этого вопрос, заслуживающий обсуждения, лучше решить вместе с командой и добавить это решение в руководство по стилю.

Существует возможность адаптировать **уже существующее руководство по стилю**, найденное в Интернете, для использования в своей команде. Например, руководства от Google являются широко известными, но также можно найти и другие подходящие варианты. Это позволя-

ет получить все преимущества наличия руководства по стилю без необ
ходимости создавать его с нуля.

Однако следует помнить, что каждая организация может адаптиро
вать правила стиля под свои нужды. Например, руководства от Googl
могут быть более консервативными в отношении новых функций языко
из-за большой кодовой базы.

Если не хочется адаптировать готовое руководство, можно **создат
свое собственное**. При возникновении дискуссий по стилю в процесс
code review можно обсудить с командой и закрепить решение в своем ру
ководстве по стилю. Хранение такого документа в системе контроля вер
сий позволяет всей команде участвовать в процессе обновления и улуч
шения стандартов.

Также возможен **гибридный подход**, который сочетает адаптаци
существующего руководства и создание собственного документа для до
полнений. Примером может служить руководство Chromium C++, кото
рое базируется на стиле C++ от Google, но имеет собственные изменени
и дополнения.

Начинайте review немедленно. Можно предположить, что автор пе
реключился на новую задачу, создал новую ветвь в системе контрол
версий Git, пока рецензент разбирает код и пишет заключение. Скоре
всего, он с нетерпением ждет этого заключения и не выполняет трудовы
функций, при этом code review приходит асинхронно к работе рецензен
та. То есть рецензент занят выполнением своих непосредственных об
занностей, но ему требуется переключится на code review, так как авто
находится в состоянии ожидания. Чем быстрее начнется code review, те
больше доверие между автором и рецензентом, при этом быстрые ответ
побуждают авторов давать на code review небольшие участки кода, чт
ускоряет итерационный процесс.

Начните с высокого уровня и спускайтесь ниже. Если на небол
шой участок кода создать гигантский документ с исправлениями, то эт
может повлиять на настроение автора и процесс внесения исправлени
затянется. При условии наличия нескольких раундов такой подход нец
лесообразен. Поэтому рекомендуется ограничить количество исправл
ний в документе, отправив автору сначала высокоуровневые замечания,
на последующих раундах добавить еще новые низкоуровневые.

Щедро используйте примеры кода. Получение хорошего кода в к
довой базе является основной функцией code review, но обучение с
трудника с целью устранения недочетов кода в будущем является так
важной задачей. Приводя в рецензии примеры кода, мы решаем пробл
мы обучения сотрудника, а качественные примеры позволяют быстр
исправлять ошибки. Кроме того, код, приведенный в результате рецензи
рования, лучше описывает проблему и решение, что показывает высоку

вовлеченность рецензента в процесс code review и создает доверитель
ные отношения.

Никогда не говорите «ты» автору. Попытки показать неверную ра
боту человека делает процесс рецензирования конфликтным и увеличи
вает вероятность создания тупика. В общем виде работа идет над улуч
шением кодовой базы, а не над доказательством ошибок и некомпетент
ности автора. В таком виде обращаться надо к коду и критиковать код, а
не его разработчика. Когда автор видит в комментарии личное обраще
ние к нему, это отвлекает его внимание от кода и переводит внимание на
личность.

Оформляйте отзывы как запросы, а не команды.

Обосновывайте принципами, а не мнениями. Когда преподаватель
указывает на ошибку словами «тут неверно сделано», студент считает,
что это личное мнение преподавателя. В случае если автор считает себя
компетентным в написании кода, отправленного на code review, то по
добные указания на ошибки будут восприниматься как личные желания
рецензента сделать кодовую базу такой, какая ему нравится, что приве
дет в тупиковой ситуации. Все замечания рецензенту следует обосновать
и при обосновании необходимо ссылаться на принципы, правила, доку
менты, а не на собственное мнение. Хотя это правило кажется вполне
естественным, но множество отдельных исправлений основывается не на
документах, а на опыте рецензента-ревьюера. Избежать тупиковой ситу
ации получится при доверии автора к рецензенту и умении рецензента
оформлять свое аргументированное мнение.

*Стремитесь повысить уровень качества кода только на одну-две
ступени.*

Ограничьте фидбек по повторяющимся примерам.

Уважайте область review.

Задача проверки кода в отрыве от остальной части программы труд
на. Часто в результате code review рецензент обнаруживает неточности
или проблемы в смежных с проверяемой областях. Такие замечания мо
гут показаться автору избыточными, как будто рецензент хочет силами
автора исправить весь код. Лучше провести рефакторинг кода в отдель
ной задаче.

Ищите возможность разбить большие review.

Искренне хвалите.

Утверждайте review, когда остались тривиальные правки.

Лучше избегайте тупиковых ситуаций.

В целом процесс code review похож на проверку лабораторных ра
бот по программированию, здесь небольшие программные модули с
определенным функционалом и конкретным заданием. Проверяешь пра
вильность и отмечаешь ошибки, пытаясь поднять уровень студента на
одну ступень, объясняешь и показываешь примеры кода и принятых ре-

шений. Основное отличие состоит в отсутствии раундов, студенты полу
чают список недочетов, и предполагается, что он исчерпывающий.

Тем не менее навык проведения грамотного review-кода, докумен
тов, отчетов и статей является важным навыком любого работника ил
руководителя. Следует практиковаться в данном деле и участвовать
рецензировании, комментировании, проведении code review своих и сто
ронних проектов.

Глава 8. ТВОРЧЕСКАЯ РАБОТА

Практическая работа в данном курсе позволяет студентам окунуться в процесс разработки сложного программного обеспечения. Обычно в вузах студенты учатся разрабатывать программное обеспечение в отрыве от реальных процессов, происходящих в больших и сложных разработках. Во-первых, программное обеспечение пишется студентом в одиночку или сдается только часть большого проекта. При назначении нескольких студентов для этих целей они разделяют приложение на независимые друг от друга части, например, frontend и backend. В результате студент взаимодействует с внешним API без возможности обсуждения и изменения этого API. Во-вторых, после завершения работы над программным обеспечением и сдачи его (и/или отчета) не развивается: ошибки не исправляются, новые изменения не вносятся, требования не пересматриваются. Студенты не участвуют в процессе эксплуатации программного обеспечения, не проводят требуемых тестов, а только отлаживают программное обеспечение для возможности сдачи преподавателю. В-третьих, при разработке программного обеспечения у студента определены четкие сроки сдачи. В-четвертых, четко определенное задание на лабораторную работу, зафиксированное на начальных этапах, обычно в начале курса обучения. В результате студенты применяют водопадную модель разработки программного обеспечения, при этом этапы анализа и постановки задачи уже проведены преподавателем. Оставшиеся этапы студент распределяет по календарному принципу. Некоторые студенты будут делать работу в свободное время, другие – ближе к времени сдачи. Любые изменения в задании и дополнительные требования преподавателя должны быть обговорены заранее, поскольку они ломают каскадную модель и календарный план работы студента и могут восприниматься студентами негативно.

Предложенная практическая работа предполагает творческий и гибкий подход в ее выполнению, изменение задания во время выполнения работы, командную разработку. Также немаловажной в процессе работы над программным обеспечением является мотивация студентов к дальнейшему развитию проекта, возможность доработки и использования данного приложения в будущем. Данный поход в рамках ограниченного времени близок к стартапам, и рекомендуется совмещать его с близкими к разработке стартапов дисциплинами.

Студенты организуются в группы для выполнения одного задания на лабораторную работу. Причем данное задание может изменяться в процессе разработки программного обеспечения.

В начале разработки программного обеспечения студенты выбирают методологию (например, DSDM (Dynamic Systems Development Method), XP (Extreme Programming), Scrum и Kanban), которая ляжет в основу по-

становки целей, определения этапов и сроков разработки. После выбор
методологии разработки необходимо распределить различные роли дл
каждого студента, определиться с сильными и слабыми сторонами. Зада
чей гибкой методологии является ранний выпуск прототипа программ
ного обеспечения с минимальным функционалом с последующим рас
ширением функционала и добавлением возможностей.

Отдельно следует отметить, что при применении гибкой методологи
предполагается совместное владение кодом проекта. То есть при защит
кода проекта преподаватель опрашивает каждого студента по всему код
(в общих чертах, основные моменты), а студенты должны изучить ко
который написали их коллеги. Для такого процесса студенты могут нап
сать документацию «для своих» и пользоваться ей при ответе преподав
телю. Часто студенты вместо написания документации пытаются личн
объяснить коллегам код, указывая на основные моменты при объяснени
Такой метод хорошо работает в маленьких командах (2–3 человека), а
больших командах рекомендуется писать самодокументированный ко
или комментарии в сложных моментах и для сложных алгоритмов. Д
сложных алгоритмов рекомендуется написание псевдокода настолько п
дробного, чтобы программист на каждую строку псевдокода знал, как
код он напишет. После этого псевдокод переводится в комментарии и п
каждый комментарий дописывается нужный код проекта.

Совместное владение кодом осуществляется не только на этапе сд
чи проекта, но и в процессе его реализации. Так, если студент, пишущ
back-end, не смог присутствовать на занятии, то за его часть кода мог
отвечать другие одногруппники из той же бригады программистов. Т
кой подход требует постоянного обучения, парного программирования
постоянной смены выполняемых задач, благодаря чему студенты глуб
окунаются в реальные процессы разработки программного обеспечен
по гибким методологиям.

Основные трудности и ошибки у студентов возникают как раз
этапе распределения ролей. Если есть студенты, не занимающимся пр
граммированием, то для них в процессе разработки будет большое кол
чество аналитических задач.

Студенты должны определиться с четким техническим заданием
первый этап для создания первого прототипа программного обеспечени
Подобная задача решается отдельным участником разработки, в обяза
ности которого входит составление документа технического задани
разделения задачи на подзадачи, создания пула задач на текущий эт
выполнения, отслеживание процессов и соответствие выполненных
дач требованиям технического задания. Кроме непосредственной пр
верки работы других членов команды разработчиков от данного чле
команды требуется составление и анализ задач на этап, отчетность

выполнению задачи, умелое декомпозирование и определение целей на
этап. В методологии SCRUM такими задачами занимается ProductOwner.

Человек, ответственный за соблюдение гибкой методологии: в ме
тодологии SCRUM за это отвечает ScrumMaster. В команде этот студент
ответственен за организацию встреч, дистанционной работы, взаимодей
ствия среди участников. Он следит за тем, чтобы встречи проходили
максимально эффективно, формирует повестку встречи, ведет протоко
лирование, резюмирует полученные результаты. В задачи студента также
входит формирование KPI, модели оценки эффективности, определение
интенсивности оценок и описание принимаемых решений при достиже
нии определенных уровней оценок. Так как возможности влияния на од
ногруппников у этого студента нет, то данные метрики должны быть
предоставлены преподавателю для оценки правильности проведения
разработки. Преподаватель не использует полученные KPI для оценки
студентов, а только помогает составлять эффективные метрики.

Предполагается также наличие разработчиков программного обес
печения. В рамках разработки современного программного обеспечения
в творческого задания предполагается создание комплексного про
граммного обеспечения, содержащего базу данных, графический интер
фейс (желательно web-интерфейс) и сложную математическую начинку
из алгоритмов.

В результате комплексного решения необходимо разделить про
граммное обеспечение на нескольких программистов, реализующих раз
личные блоки программного обеспечения. Простой вариант заключается
в разделении программного обеспечения (и студентов-разработчиков) на
front-end и back-end. Такое разделение возможно, если студентов-
разработчиков двое или трое, но если в команде 5–6 студентов-
разработчиков, то необходимо разделить программную реализацию. Та
кой подход возможен при применении хороших архитектурных шабло
нов и знании объектно-ориентированного программирования. Примене
ние архитектурного подхода MVC (с различными производными и по
добными архитектурами: MTV, MVP, MVVM) приводит к разделению
программного обеспечения на две части и, как следствие, к задействова
нию только двух программистов.

Применение более сложных архитектурных шаблонов требует уме
ния оперировать с абстракциями. Программное обеспечение можно раз
делить на следующие составляющие:

- данная модель;
- алгоритмические вычисления;
- альтернативные алгоритмы;
- дополнительные, параллельные вычисления;
- Front-end код;

– загрузчик/генератор данных;
– модуль unit-тестирования;
– взаимодействие с внешним API.

Работа с кешированием результатов, специализированным ПО и базами данных.

Возможно, потребуются алгоритмисты, студенты, которые будут составлять, анализировать и подбирать параметры сложных эвристических или оптимизационных алгоритмов. При применении систем искусственного интеллекта, машинного обучения, работы нейронных систем необходимы специалисты по ML, специалисты по DataScience. Данные студенты должны предложить, запрограммировать и обеспечить работу с BigData.

Отдельно следует отметить специалиста по CI/CD и деплою кода на сервер продакшн. Это может быть как тестовый или внутренний сервер в рамках учебной организации, так и реальный арендованный на короткий срок сервер. Специалист обеспечивает работу сервера, бесшовное обновление версий, описывает процедуры вертикального и горизонтального масштабирования, изучает и применяет техники профессии DevOps.

Для взаимодействия программистов и управления задачами студенты пользуются системой управления проектами (рекомендуется система Yougile). В этой системе создаются задачи (таски, task) которые необходимо выполнять студентам. Такая система позволяет создавать и фиксировать идеи, пользовательские истории, замечания преподавателя, ошибки и трудности, возникающие у студентов. При взаимодействии с преподавателем на занятиях студенты оперируют системой управления проектами, определяя текущее состояние выполнения задач, например, что было сделано после собрания, осуществление планов и т. д. Работа с системой управления проектами приучает студентов к работе в команде, приспосабливает к взаимодействию с коллегами в серьезных больших проектах.

Хранение кода осуществляется в системе контроля версий. Предполагается использование систем контроля версий Git с реализациями GitHub или GitLab, но возможны и другие варианты. Обычно некоторые студенты имеют опыт применения системы контроля версий Git, но используют ее как облачное хранилище кода. В рамках данной работы репозиторий используется для взаимодействия и написания кода одновременно несколькими программистами. Идеально использование pull request для создания задач и управления процессом модификации и написания кода программного обеспечения. Для лучшего понимания процессов работы с системой контроля версий Git далее представлена специальная лабораторная работа.

Отдельно следует указать обязательное применение методов CI/CD для доставки программного обеспечения и развертывания его даже на тестовом сервере (localhost). Изучение данных методов позволяет студентам работать с unit-тестами и разобраться в том, как разворачивать приложение на реальном сервере. При использовании системы контроля версии Git на основе GitHub CI/CD может выполняться средствами GitHub, специальной настройкой системы контроля версий.

Для совместного использования кода студентам необходимо проводить review кода своих одногруппников. Такой подход позволяет студентам изучать дополнительные технологии, и при этом не существует кода, который пишет (и разбирается в нем) только один студент. Review кода необходимо проводить перед каждой встречей с преподавателем и по результатам review писать небольшой ответ. Чтобы review не требовал большого времени, в рамках review на каждую встречу необходимо рассмотреть не более 100 строк кода. Желательно, чтобы к встречам каждый студент делал review чужого кода, таким образом, за время прохождения курса review каждый студент минимально хотя бы раз выполнит review кода.

Рассмотрим примеры заданий.

Задание 1. «Русский язык» – разработать программу (комплекс программ), позволяющую проанализировать текст на русском языке, разбив каждое предложение по членам предложения.

Входными данными программы является текстовый файл (.txt и .doc).

Результат работы программы выводится на экран в следующем виде:

– выводится весь текст с выделенными отдельными членами предложения;
– по запросу пользователя есть возможность выделения только отдельных членов предложения;
– предусмотрена возможность вывода статистики по частоте встречаемости различных слов в качестве соответствующих членов предложения;
– просматривается содержимое базы данных;
– результаты анализа необходимо хранить в базе данных, реализованной на выбранной СУБД.

Особенности программы:

– распознавание текста должно быть точнее 75%;
– необходимо обеспечить приемлемое быстродействие работы программы;
– программное приложение должно быть разработано по методологии MVC.

Желательно рассмотреть по согласованию с преподавателем другие функциональные возможности разрабатываемого ПО.

Задание 2. «Интермодальные перевозки» – разработать программу (комплекс программ), позволяющую получить множество маршрутов при интермодальных перевозках. Рассматриваются интермодальные перевозки с использованием различных видов транспорта, в нашем случае – пассажирского.

Особенности программы:

- программа должна позволять за приемлемое время находить множество путей из выбранных пунктов A и B;
- при разработке программы рассмотреть следующие виды транспорта: самолет, поезд, автобус и машина, и в работе рассмотреть около 1 000 пунктов и 5 000 – 10 000 маршрутов между городами;
- следует учитывать расписание, т. е. время прибытия и отправления;
- приемлемое время выдачи первого маршрута – 30 с;
- рассмотреть пользовательский интерфейс с возможностью выбора пунктов отправления и назначения и визуализации маршрутов;
- учитывать различные ограничения: отсутствие прав, не летает на самолетах, на конкретных компаниях, не ездит на поездах;
- обеспечить сортировку результатов работы по стоимости, времени и др.;
- время в пути, отправления и прибытия стараться брать приближенным к реальности.

Задание 3. «Маршруты» – в составе бригады главного программиста разработать программу (комплекс программ), позволяющую вычислять маршруты инкассаторов по Москве. На территории города расположено множество банкоматов. В каждом банкомате имеются два бункера: один – для приема, другой – для выдачи денег. Задача на каждый день – рассчитать маршрут для нескольких групп инкассаторов так, чтобы не было ситуации, в которой бункер переполнился для приема или опустел для выдачи. При расчете показателей для каждого банкомата имеются статистические характеристики (математическое ожидание и дисперсия) количества банкнот, поступающих или извлекаемых из банкомата, и размеры его бункера. В нашем случае банкоматы имеют различные размеры бункеров и различные значения характеристик, но будем считать, что каждый день эти характеристики одинаковые.

Особенности программы:

- количество групп инкассаторов – 5;
- количество банкоматов – 1 000;
- маршрут надо уметь строить на несколько дней вперед;

- перемещение между банкоматами привязать к дорожной сети; время перемещения между банкоматами задано статистическим характеристиками (математическое ожидание и дисперсия);
- рабочий день инкассаторов – не более 8 часов;
- необязательно каждый день объезжать все банкоматы;
- на экран следует выводить информацию о маршрутах инкассаторов на каждый запрошенный день.

Глава 9. ПРАКТИЧЕСКИЕ РАБОТЫ

9.1. Практическая работа № 1. Система контроля версий Git

Создайте новый Remote Repository (удаленный репозиторий) на GitHub. Укажите описание репозитория. Произведите клонирование репозитория на локальную машину в Local Repository и создайте LocalWorkingDirectory.

В папке LocalWorkingDirectory (с названием person1) создайте приложение с 3 файлами на любом языке программирования (возможно работа с текстовыми файлами). Добавьте Commit (коммит) в Local Repository (локальный репозиторий), перенесите Commit в Remote Repository.

Произведите клонирование Remote Repository еще в одну LocalWorkingDirectory (с названием person2). В текущей LocalWorkingDirectory добавьте в один из файлов новую информацию, добавьте Commit в Local Repository. Проверьте изменения в LocalWorkingDirectory person1. Загрузите изменения из LocalWorkingDirectory person2 в Remote Repository, скачайте изменения в LocalWorkingDirectory person1, проверьте изменения в LocalWorkingDirectory person1. Добавьте изменения в другой файл, создайте новый Commit и повторите работу с Remote Repository.

Создайте новые ветви. Для разработки новой функциональности необходимо создать новую ветвь в Local Repository person1 с названием dev. Переключитесь на нее. Создайте новый файл в LocalWorkingDirectory person1 в новой ветви. Сделайте новый Commit в ветви и проведите обновление в Remote Repository.

В LocalWorkingDirectory person2 получите последние обновления из Remote Repository, создайте новую ветвь dev, в которой создайте новый файл, выполните Commit и загрузите его в Remote Repository. Сделайте небольшие изменения в новом файле и добавьте эти изменения в предыдущий Commit.

Изучите слияние ветвей с помощью Squash Commits, Merge, Rebase. Приведите примеры слияния и полученные результаты. Слияние ветвей проведите повторно, выполняя создание новых ветвей.

Разрешение конфликтов слияния. Создайте третью LocalWorkingDirectory person3 и загрузите туда копию из Remote Repository. Для третьей LocalWorkingDirectory создайте свою ветвь dev3 и измените в ней один из первых файлов с последующим созданием Commit и выгрузкой Commit на Remote Repository. Также необходимо изменить другой файл в LocalWorkingDirectory person2 в ветвь dev2, создать и выгрузить новый Commit на Remote Repository. В LocalWorkingDirectory person1 необходимо изменить тот же файл, что и в третьей LocalWorkingDirectory, но в другом месте, и добавить Commit в ветвь dev1. Данные действия имитируют работу трех программистов над разными задачами и одинаковыми или различными файлами проекта. Поочередно проведите слияние ветви dev с ветвью master и опишите результаты. Рассмотрите методы исправления конфликтов слияния.

Используйте git log для нахождения хэша Commit. Откатите изменения до выбранного Commit с помощью git reset --hard <commit_hash>. Для просмотра изменений в каждом Commit можно использовать git log –p.

Используйте команду git cherry-pick <commit_hash> для перемещения Commit из ветви dev2 на текущую ветвь. Используйте git rebase для перебазирования Commit ветвь dev1 на другую ветвь. Для перемещения нескольких Commit можно использовать git rebase -i <commit_hash>. Создайте серию коммитов в ветвь dev3, создайте патчи для этих коммитов с помощью git format-patch и примените их к ветви main с помощью git apply.

Вариант 1. «Создание тегов для версионирования кода»:

Используя команду git tag <tag_name>, создайте легковесный тег для текущего состояния репозитория.

Проверьте созданный тег с помощью команды git tag.

Для создания аннотированного тега с сообщением используйте команду git tag -a <tag_name> -m "Message".

Просмотрите список всех тегов с помощью команды git tag -l.

Публикуйте созданный тег на удаленном репозитории с помощью команды git push origin <tag_name>.

Убедитесь, что тег успешно опубликован, проверив его на удаленном репозитории.

Создайте еще один легковесный тег с другим именем с использованием git tag.

Просмотрите изменения в тегах после создания нового тега с помощью git tag -l.

Отправьте оба тега на удаленный репозиторий с помощью команды git push origin –tags.

Проверьте наличие всех тегов на удаленном репозитории с помощью git ls-remote –tags origin.

Вариант 2

1. Добавьте подмодуль: используйте команду git submodule add <repository_url> <path> для добавления подмодуля из удаленного репозитория в основной репозиторий.

2. Просмотрите список подмодулей: можно использовать команду git submodule для просмотра списка добавленных подмодулей.

3. Обновление подмодуля: чтобы обновить содержимое подмодуля до последней версии из его репозитория, используйте команду git submodule update --remote.

4. Переключение на определенную версию подмодуля: можно переключиться на определенную версию подмодуля, указав хэш коммита или тег.

5. Изменение URL удаленного репозитория подмодуля: для изменения URL удаленного репозитория подмодуля отредактируйте файл .gitmodules и используйте команду git submodule sync.

6. Инициализация и обновление подмодулей после клонирования репозитория: после клонирования репозитория с подмодулями выполните git submodule init для инициализации подмодулей и git submodule update для обновления их содержимого.

7. Просмотр изменений в подмодуле: можно просмотреть изменения в подмодуле, перейдя в его директорию и использовав стандартные команды Git (например, git status, git diff).

8. Удаление подмодуля: для удаления подмодуля удалите соответствующую секцию в файле .gitmodules и используйте команду git rm --cached <path>.

9. Клонирование репозитория с подмодулями: при клонировании репозитория с подмодулями, используйте флаг --recurse-submodules, чтобы клонировать, инициализировать и обновить подмодули.

10. Проверка статуса подмодулей: для проверки статуса подмодулей используйте команду git submodule status.

Вариант 3. «Использование стэшей для временного сохранения изменений»:

Создайте новый файл или внести изменения в существующий файл в рабочем каталоге.

Используйте команду git stash save "message" для сохранения временных изменений в стэш с сообщением.

Проверьте список стэшей с помощью команды git stash list и убедитесь, что сохранение прошло успешно.

Внесите дополнительные изменения в рабочий каталог.

Примените последний стэш к рабочему каталогу с помощью команды git stash apply.

Проверьте, что изменения из стэша успешно применены к рабочему каталогу.

Продолжите работу над проектом и внесите дополнительные изменения.

Примените последний стэш к рабочему каталогу и удалите его из списка стэшей с помощью команды git stash pop.

Завершите работу над проектом, закоммитите все изменения и отправьте их на удаленный репозиторий.

Вариант 4. «Использование хуков для автоматизации процессов»:

Откройте терминал и перейдите в рабочую директорию вашего Git-репозитория.

Создайте папку .git/hooks/, если она еще не существует.

В папке .git/hooks/ создайте скрипты для автоматического выполнения определенных действий при определенных событиях Git. Например, вы можете создать скрипт pre-commit, который будет запускаться перед коммитом.

Напишите скрипт, который будет выполнять нужные вам действия при наступлении определенного события Git. Создать скрипт, который будет проверять форматирование python-кода с помощью инструмента black перед каждым коммитом. Создать скрипт, который будет запускать тесты перед коммитом.

Сохраните скрипт с правильным именем (например, pre-commit) без расширения файла в папке .git/hooks/.

Сделайте скрипт исполняемым с помощью команды chmod +x .git/hooks/pre-commit.

Протестируйте работу хука, сделав изменения в файле и попытавшись закоммитить их.

Убедитесь, что скрипт успешно выполняется при наступлении заданного события Git.

Вариант 5. «Использование submodules для управления зависимостями проекта»:

Добавьте библиотеку для работы с базой данных (например, PostgreSQL) в проект с использованием Git submodules.

Добавьте библиотеку для работы с PostgreSQL в проект как submodule. Для необходимости выполните команду: git submodule add <URL_репозитория_библиотеки> lib/postgresql.

Выполните команду git submodule update --init --recursive, чтобы склонировать репозиторий библиотеки и подключить его к вашему проекту.

Проверьте документацию библиотеки и настройте ее в соответствии с вашими потребностями.

Импортируйте необходимые модули из библиотеки в ваш код и начните использовать ее функционал для работы с базой данных PostgreSQL.

Если вам нужно внести изменения в саму библиотеку, перейдите в директорию submodule (lib/postgresql) и работайте над ней как отдельным репозиторием.

После внесения изменений в submodule выполните команды git add . и git commit -m "Сообщение коммита", чтобы зафиксировать изменения.

Перейдите обратно в корневую директорию вашего проекта и выполните git add lib/postgresql и git commit -m "Обновление библиотеки PostgreSQL", чтобы зафиксировать обновление submodule.

При необходимости отправьте изменения в удаленный репозиторий проекта и submodule с помощью git push.

Если другие разработчики работают над проектом, они могут обновить submodule, выполнив git submodule update --remote, чтобы получить последнюю версию библиотеки.

Вариант 6. «Работа с алиасами для упрощения команд Git»:

Откройте терминал и выполните команду nano ~/.gitconfig для открытия файла конфигурации Git в текстовом редакторе Nano.

В файле конфигурации Git добавьте следующие строки для создания алиасов:

```
[alias]
st = status
ci = commit
co = checkout
br = branch
df = diff
lg = log --oneline --decorate --all --graph
```

Это пример алиасов для упрощения команд: status, commit, checkout, branch, diff и log.

Сохраните изменения в файле конфигурации Git, нажав Ctrl + O, затем Enter, и выйдите из редактора, нажав Ctrl + X.

Перезапустите терминал и выполните команды git st, git ci, git co, git br, git df и git lg для проверки, что алиасы работают корректно.

Создайте алиас git recent для команды git log --oneline -n 5 и проверьте его работоспособность.

Измените или удалите существующие алиасы, отредактируйте файл конфигурации Git соответственно.

Для удобства работы команды Git с алиасами рекомендуется документировать их использование в README вашего проекта или в специальном файле с описанием.

Обсудите созданные алиасы с другими участниками проекта, чтобы все могли использовать их.

Вариант 7. «Использование Git bisect для поиска ошибок в истории Commit»:

Рассмотрите работу программы с ошибкой, например, деление на 0. Запустите процесс поиска ошибки в программном коде с помощью команды git bisect start. Выберите известный хороший Commit, где ошибка отсутствует, и пометьте его с помощью команды git bisect good <Commit ID>.

Выберите известный плохой Commit, где ошибка проявляется, и пометьте его с помощью команды git bisect bad <Commit ID>. После указания хорошего и плохого Commit проверьте текущее состояние проекта с помощью команды git bisect visualize, чтобы убедиться, что процесс запущен корректно. Git автоматически перейдет к промежуточному Commit между хорошим и плохим и затем перейдет к следующему промежуточному Commit для дальнейшего анализа. Повторяйте эти шаги 5–6 раз до тех пор, пока не будет найден проблемный Commit.

Когда будет найден проблемный Commit, завершите процесс поиска ошибки с помощью команды git bisect reset. Исправьте найденную ошибку в проблемном Commit и закоммитьте изменения.

После исправления ошибки убедитесь, что проблема больше не возникает, и продолжайте работу над проектом.

Вариант 8. «Работа с Git reflog для восстановления удаленных или потерянных Commit»:

Запустите Git reflog, чтобы просмотреть историю действий ветвями.

Переместитесь на любой предыдущий коммит. Используйте команду git reset --hard HEAD~1. Это удалит последний коммит и все изменения, внесенные в него.

Изучите вывод Git reflog, чтобы найти удаленный коммит или изменения, которые были потеряны. Скопируйте хэш удаленного коммита, который вы хотите восстановить.

Создайте новую ветвь на основе хэша удаленного коммита с помощью команды git checkout -b <branch_name> <commit_hash>. Переключитесь на новую ветвь и убедитесь, что удаленный коммит успешно восстановлен.

Просмотрите историю изменений с помощью Git log, чтобы убедиться, что удаленный коммит или потерянные изменения были успешно восстановлены.

Закоммитите восстановленные изменения. Убедитесь, что ваш репозиторий находится в актуальном состоянии и все изменения сохранены.

Вариант 9. «Использование Git LFS (Large File Storage) для управления большими файлами»:

Установите Git LFS на ваш компьютер, если он еще не установлен, с помощью инструкций на официальном сайте Git LFS.

Добавьте поддержку Git LFS к существующему репозиторию с помощью команды git lfs install.

Добавьте несколько больших файлов в репозиторий с помощью команды git lfs track "*.extension", где extension – расширение файлов, которые вы хотите отслеживать.

Сделайте Commit с добавленными большими файлами с помощью команды git add . и git commit -m "Added large files".

Проверьте статус больших файлов с помощью команды git lfs ls-files, чтобы убедиться, что они правильно отслеживаются Git LFS.

Отправьте изменения на удаленный репозиторий.

Скачайте репозиторий на другой компьютер и убедитесь, что большие файлы успешно загружены с помощью Git LFS.

Попробуйте изменить настройки отслеживания файлов в Git LFS, например, добавить новые типы файлов для отслеживания.

Удалите один из больших файлов из репозитория с помощью команды git rm --cached filename и проверьте, что он больше не отслеживается Git LFS.

После успешного использования Git LFS для управления большими файлами завершите работу, убедитесь, что все изменения сохранены, и продолжайте разработку проекта с использованием Git LFS для эффективного управления большими файлами.

9.2. Практическая работа № 2. CI/CD

Настройте систему непрерывной интеграции (CI) для вашего проекта, например, используя сервисы GitHub Actions, GitLab CI/CD, Jenkins, Travis CI или другие инструменты. Пример «Настройка CI с помощью GitHub Actions»:

Откройте репозиторий проекта на GitHub. Перейдите во вкладку Actions в верхней части страницы репозитория. Нажмите на кнопку Set up a workflow yourself или выберите один из шаблонов для создания нового workflow.

Создайте файл с названием main.yml в папке .github/workflows/ вашего репозитория и определите в нем шаги вашего CI/CD процесса.

Сделайте коммит и отправьте изменения в ваш репозиторий на GitHub.

Проверка работы CI: после отправки изменений в репозиторий перейдите во вкладку Actions на GitHub. Отобразите в отчете список запущенных workflow и их статус. Проверьте логи выполнения шагов вашего CI/CD-процесса.

Настройте CI таким образом, чтобы он автоматически запускался при каждом новом коммите в ваш репозиторий.

Добавьте скрипты для тестирования вашего проекта (например, unit-тесты, интеграционные тесты) и убедитесь, что они успешно проходят при запуске через CI.

Создайте ветвь development для разработки новых функций и исправлений.

Разработайте новую функцию или исправление ошибки в вашем проекте и закоммитите изменения в ветвь development.

Убедитесь, что CI успешно запустился после коммита в ветвь development и прошел все тесты.

Создайте Pull Request (PR) для вливания изменений из ветви development в основную ветвь (например, main).

Настройте автоматическое тестирование и слияние PR при успешном прохождении всех тестов.

Проверьте, что изменения из ветви development успешно влились в основную ветвь с помощью CI/CD.

Создайте ветвь release для подготовки к выпуску новой версии вашего проекта.

Подготовьте все необходимые изменения (например, обновление версии, исправление багов) в ветвь release.

Убедитесь, что CI успешно запустился после коммита в ветвь release и все тесты прошли успешно.

Создайте новый тег для версии вашего проекта и опубликуйте его на платформе Git.

Создайте документацию к вашему проекту и добавьте ее в репозиторий.

Настройте автоматическую генерацию документации при каждом обновлении кода через CI/CD.

Проверьте, что документация успешно обновляется при каждом изменении кода.

Заключите текущую версию проекта и завершите работу над лабораторной работой.

9.3. Практическая работа № 3. Руководство по стилю и code review

При выполнении практической работы студенты разрабатывают руководство по стилю для реализации командного взаимодействия и проводят code review. Руководство по стилю может быть основано на известных сторонних руководствах, так и на принятых в определенном языке программирования нормах и парадигмах.

В рамках первого этапа практического задания необходимо совместными усилиями разработать руководство по стилю для бригады программистов.

Пункты руководства по стилю определяются в соответствии с вариантом, при этом необходимо сделать все пункты/разделы из варианта.

Вариант 1

Определение стандартов и принципов стиля кодирования.

Применение логических операторов и условий для улучшения читаемости кода.

Соблюдение принципов DRY (Don't Repeat Yourself) и KISS (Keep It Simple, Stupid).

Работа с базами данных и обработка запросов к ним.

Вариант 2

Использование понятных и информативных имен переменных, функций и классов.

Использование модульного подхода к разработке программного обеспечения.

Применение SOLID-принципов при проектировании классов и объектов.

Работа с внешними API и сервисами.

Вариант 3

Форматирование кода с использованием отступов и переносов строк.

Разделение функционала на независимые модули или компоненты.

Управление зависимостями и версиями библиотек.

Обновление и поддержка программного обеспечения после выпуска.

Вариант 4

Использование комментариев для пояснения сложных участков кода.

Проверка типов данных и корректности ввода данных.

Работа с параллельными процессами и многопоточностью.

Рефакторинг кода для улучшения его читаемости, эффективности и поддерживаемости.

Вариант 5

Поддержка единообразного формата файлов и структуры проекта.

Документирование API и интерфейсов для удобной работы с кодом.

Обработка асинхронных операций и событий.

Обработка и логирование ошибок для облегчения отладки и исправления проблем.

Вариант 6

Обработка ошибок и исключений согласно установленным правилам.

Использование стандартных библиотек и инструментов разработки.

Использование паттернов проектирования для улучшения архитектуры программы.

Управление версиями кода с использованием систем контроля версий, таких как Git.

Вариант 7

Избегание излишней вложенности и длинных цепочек вызовов методов.

Обеспечение безопасности кода и защиты от уязвимостей.

Разработка документации для кода, проекта и пользователей.

Использование контейнеризации и оркестрации для управления развертыванием приложений.

Вариант 8

Тестирование кода для обеспечения его надежности и стабильности.

Локализация и интернационализация приложения.

Разработка интерфейсов пользователя с учетом принципов юзабилити и UX-дизайна.

Работа с асимметричными и симметричными шифровальными алгоритмами.

Вариант 9

Оптимизация производительности кода и использование эффективных алгоритмов.

Поддержка межплатформенности и межъязыковой совместимости.

Создание механизмов авторизации и аутентификации пользователей.

Работа с асинхронными API и обработка запросов в реальном времени.

На втором этапе студенты должны провести code review. Выбирается небольшой участок кода. Рекомендуется, чтобы данный участок кода исправлял автор, но если студент не писал код проекта, то можно выбрать несложный небольшой (150–200 строк кода), обязательно разобравшись в нем. Выбранный участок кода отдается на code review другому студенту, который проверяет написанный код и предлагает исправления и улучшения. Все предложенные исправления и взаимодействие между студентами должны быть оформлены в отчете. Например, если общение происходит средствами почты или системы управления проектами (что является наиболее распространенной практикой), то необходимо предоставить тексты писем, если это устный code review, то приводится стенограмма разговоров. Минимум необходимо провести 2 раунда code review с выявлением исправлений и рекомендаций в каждом раунде.

При проведении code review следует руководствоваться рекомендациями из соответствующей главы методического пособия.

9.4. Практическая работа № 4. TDD

Это практическое задание должно познакомить студентов с принципами автоматического и непрерывного тестирования. Для знакомства лучше всего подойдут принципы TDD (test-driven development, разработка через тестирование), в основе которых – работа с циклом: написание теста, написание кода, рефакторинг. При этом предполагается, что написанный код должен самым простым образом выполнить все созданные тесты и ничего более, а при написании тестов следует руководствоваться правилом «от простого к сложному». Итерация такого подхода занимает до 5 минут, и поэтому удается быстро наполнить тестовую базу. В рамках практического занятия студенты проводят 15 итераций разработки части программного обеспечения с применением методологии TDD. Данную часть программного обеспечения и созданные автоматические тесты позднее предполагается внедрить в общую разработку. В случае если разработка модулей для творческого проекта не представляется возможным, предлагаются следующие варианты заданий:

Вариант 1. Разработка простого калькулятора с использованием TDD. Задание: разработать простой калькулятор, который поддерживает операции сложения, вычитания, умножения и деления.

Вариант 2. Разработка приложения для управления списком задач с использованием TDD. Задание: разработать приложение для управления списком задач (to-do list), которое позволяет добавлять, удалять и отмечать задачи как выполненные.

Вариант 3. Разработка системы авторизации и аутентификации с использованием TDD. Задание: разработать систему авторизации и аутентификации пользователей, которая позволяет регистрироваться, входить в систему и управлять учетной записью.

Вариант 4. Разработка API для управления пользователями с использованием TDD. Задание: разработать API для управления пользователями, который позволяет создавать, обновлять, удалять и получать информацию о пользователях.

Вариант 5. Разработка игры крестики-нолики с использованием TDD. Задание: разработать простую игру крестики-нолики.

Вариант 6. Разработка веб-приложения для управления задачами с использованием TDD. Задание: разработать веб-приложение для управления задачами (to-do list) с использованием фреймворка Django или Flask.

Вариант 7. Разработка системы оплаты онлайн-магазина с использованием TDD. Задание: разработать систему оплаты для онлайн-

магазина, которая позволяет пользователям оформлять заказы и производить оплату.

Вариант 8. Разработка мобильного приложения на React Native с использованием TDD. Задание: разработать мобильное приложение на React Native, которое представляет собой список задач (to-do list) с возможностью добавления и удаления задач.

Вариант 9. Разработка системы управления складом с использованием TDD. Задание: разработать систему управления складом, которая позволяет добавлять новые товары, изменять их количество на складе и отслеживать поступление и отгрузку товаров.

9.5. Практическая работа № 5. Рефакторинг кода

При выполнении практической работы студенты учатся проводить рефакторинг существующего кода, кода программного обеспечения, созданного в результате выполнения творческой работы. Отчетностью по данной практической работе служит измененный код с указанием «было-стало». Для каждого изменения дается словесное описание причин изменения и достоинств, полученных в результате изменения кода. Также требуется дать словесную (числовую, приблизительную оценку) технического долга для функции/объекта/модуля, в которых произведен рефакторинг. При изучении рефакторинга важно рассмотреть различные варианты изменения кода. Так как программное обеспечение, написанное студентами, обычно не обладает сложными зависимостями, то студенты будут стремиться выполнить в результате рефакторинга только простые методы по типу «Переименование метода (Rename Method)». В результате рефакторинга студентам разрешается брать способы только из тех, что приведены в вариантах. При этом необязательно использовать все способы рефакторинга, а можно многократно повторить один из них.

Вариант 1

Выделение метода (Extract Method).

Замена метода объектом методов (Replace Method with Method Object).

Удаление посредника (Remove Middle Man).

Инкапсуляция поля (Encapsulate Field).

Вариант 2

Встраивание метода (Inline Method)

Параметризация метода (Parameterize Method)

Замена условного оператора полиморфизмом (Replace Conditional with Polymorphism).

Удаление метода установки значения (Remove Setting Method).

Вариант 3

Введение поясняющей переменной (Introduce Explaining Variable).

Сокрытие делегирования (Hide Delegate).

Замена однонаправленной связи двунаправленной (Change Unidirectional Association to Bidirectional).

Инкапсуляция нисходящего преобразования типа (Encapsulate Downcast).

Вариант 4

Замена временной переменной вызовом метода (Replace Temp with Query).

Замена массива объектом (Replace Array with Object).

Замена кода типа состоянием/стратегией (Replace Type Code with State/Strategy).

Замена кода ошибки исключительной ситуацией (Replace Error Code with Exception).

Вариант 5

Выделение интерфейса (Extract Interface).

Подъем метода (Pull Up Method).

Самоинкапсуляция поля (Self Encapsulate Field).

Удаление присваиваний параметрам (Remove Assignments to Parameters).

Вариант 6

Удаление управляющего флага (Remove Control Flag).

Инкапсуляция коллекции (Encapsulate Collection).

Разделение запроса и модификатора (Separate Query from Modifier).

Замена конструктора фабричным методом (Replace Constructor with Factory Method).

Вариант 7

Замещение алгоритма (Substitute Algorithm).

Замена магического числа символической константой (Replace Magic Number with Symbolic Constant).

Замена кода типа подклассами (Replace Type Code with Subclasses).

Декомпозиция условного оператора (Decompose Conditional).

Вариант 8

Введение поясняющей переменной (Introduce Explaining Variable).

Замена условного оператора полиморфизмом (Replace Conditional with Polymorphism).

Введение граничного объекта (Introduce Parameter Object).

Встраивание временной переменной (Inline Temp).

Вариант 9

Дублирование видимых данных (Duplicate Observed Data).

Разделение запроса и модификатора (Separate Query from Modifier).

Сокрытие делегирования (Hide Delegate).

Замена однонаправленной связи двунаправленной (Change Unidirectional Association to Bidirectional).

9.6. Практическая работа № 6. Архитектуры и паттерны

Практическая работа нацелена на изучения основ проектирования архитектуры, изменения, модернизации кода для удобства модификации и понимания кода программного обеспечения. В рамках практической работы необходимо составить архитектуру (структуру) разрабатываемого в творческой работе программного обеспечения, выделить отдельные компоненты, блоки, указать паттерны проектирования. В отчете по практической работе рассмотреть архитектурные решения с точки зрения принципов SOLID. Кроме составленной схемы, которая может быть составлена коллективом разработчиков или выполняться в рамках творческого задания (отдельных task в системе управления проектами), студентам необходимо предложить модернизацию программного кода путем добавления необходимых паттернов по вариантам:

- Добавление паттерна «фабрика».
- Добавление локатора сервисов.
- Добавления наблюдателя или медиатора/посредника.
- Добавление паттерна «стратегия».
- Изменить код программного обеспечения для выполнения принципа подстановки Барбары Лисков (Liskov Substitution Principle).
- Изменить код программного обеспечения для выполнения принципа инверсии зависимостей (Dependency Inversion Principle).
- Добавление паттерна «фасад».
- Добавление паттерна «цепочка обязанностей»
- Добавление паттерна «строитель».

Список литературы

1. *Басс Л., Клементс П., Кацман Р.* Архитектура программного обеспечения на практике. – 2-е изд. – СПб. : Питер, 2006.

2. *Бек К.* Экстремальное программирование: разработка через тестирование. Библиотека программиста. – СПб. : Питер, 2003.

3. *Вигерс К.* Разработка требований к программному обеспечению. – М. : Русская редакция, 2004.

4. *Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж.* Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб. : Питер, 2021.

5. *Демарко Т.* Deadline. Роман об управлении проектами. – М. : Вершина, 2006.

6. *Кон М.* Agile: оценка и планирование проектов. – М. : Альпина Паблишер, 2018.

7. *Мартин Р.* Чистая архитектура. Искусство разработки программного обеспечения. – СПб. : Питер, 2022.

8. *Нильссон Дж.* Применение DDD и шаблонов проектирования. Проблемно-ориентированное проектирование приложений с примерами на C# и .NET. – Вильямс, 2008.

9. *Ричардсон К.* Микросервисы. Паттерны разработки и рефакторинга. – СПб. : Питер, 2022.

10. *Рубин К. С.* Основы Scrum: практическое руководство по гибкой разработке ПО. – М. : И. Д. Вильямс, 2016.

11. *Спинеллис Д., Гусиос Г.* Идеальная архитектура. Ведущие специалисты о красоте программных архитектур / пер. с англ. – СПб. : Символ-Плюс, 2010.

12. *Фарли Д.* Современная программная инженерия. ПО в эпоху эджайла и непрерывного развертывания. – СПб. : Питер, 2023.

13. *Фаулер М.* Архитектура корпоративных программных приложений. – М. : И. Д. Вильямс, 2006.

14. *Фаулер М.* Рефакторинг: улучшение существующего кода. – СПб. : Символ-Плюс, 2003.

15. *Форд Н., Парсонс Р., Куа П.* Эволюционная архитектура. Поддержка непрерывных изменений. – СПб. : Питер, 2022.

16. *Шор Дж., Уорден Ш.* Искусство agile-разработки. Теория и практика гибкой разработки ПО. – СПб. : Питер, 2024.

17. *Atchison L.* Architecting for Scale. – 2020.

18. *Hunt A., Thomas D.* The Pragmatic Programmer: Your Journey to Mastery. – 2019.

19. *Newman S.* Monolith to Microservices. Evolutionary Patterns to Transform Your Monolith. – 2019.

20. *Scott C.* Pro Git (Expert's Voice in Software Development). – Apress, 2009.

Учебное издание

ТИТОВ Юрий Павлович

ТЕХНОЛОГИИ КОЛЛЕКТИВНОЙ РАЗРАБОТКИ ИНФОРМАЦИОННЫХ СИСТЕМ

Учебное пособие

Редактор *Н. В. Пятосина*
Оформление обложки *Ю. С. Жигалова*

Подписано в печать 03.09.2024. Формат 60x84 1/16.
Усл. печ. л. 6. Уч.-изд. л. 7,12.
Тираж 50 экз. Заказ № 84.

ФГБОУ ВО «РЭУ им. Г. В. Плеханова».
109992, Москва, Стремянный пер., 36.
Напечатано в ФГБОУ ВО «РЭУ им. Г. В. Плеханова».
109992, Москва, Стремянный пер., 36.